Training Large Language Models
SFT, PEFT, RLHF, and all that jazz

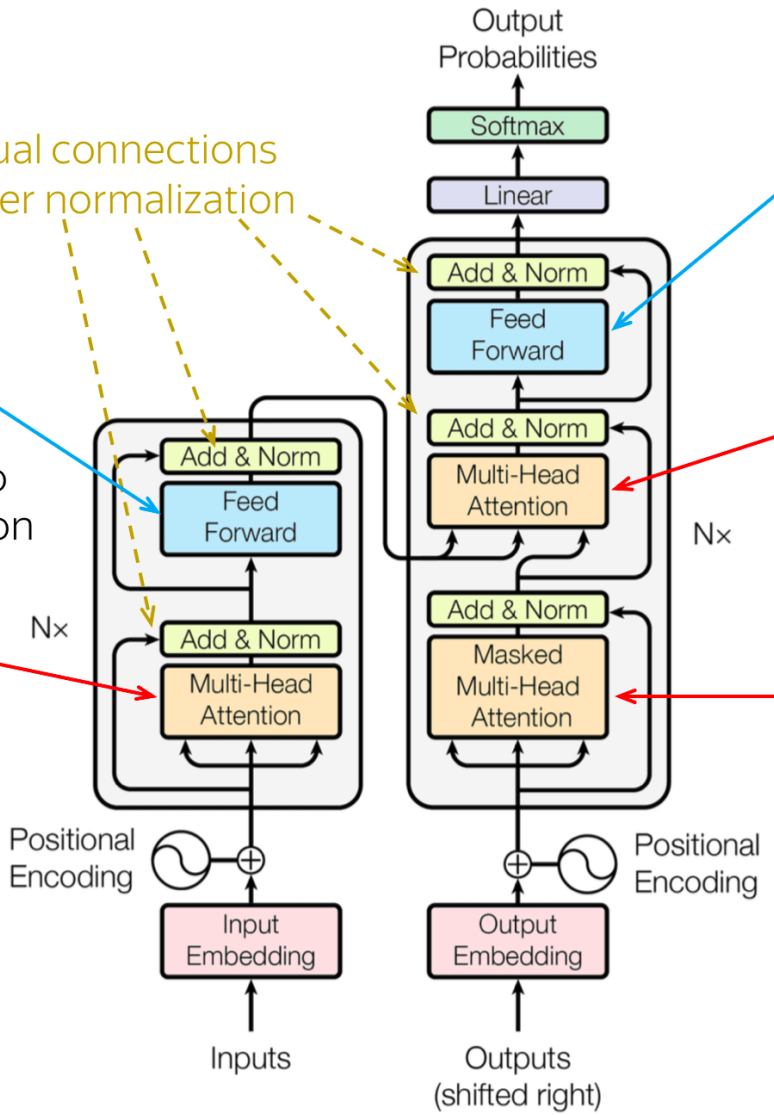
Image credit: lena-voita.github.io and the respective papers

Yandex Research

LAMBDA

British Hedgehog Preservation Society

Recap: Transformers



Residual connections and layer normalization

Feed-forward network: after taking information from other tokens, take a moment to think and process this information

Encoder self-attention: tokens look at each other

queries, keys, values are computed from encoder states

Feed-forward network: after taking information from other tokens, take a moment to think and process this information

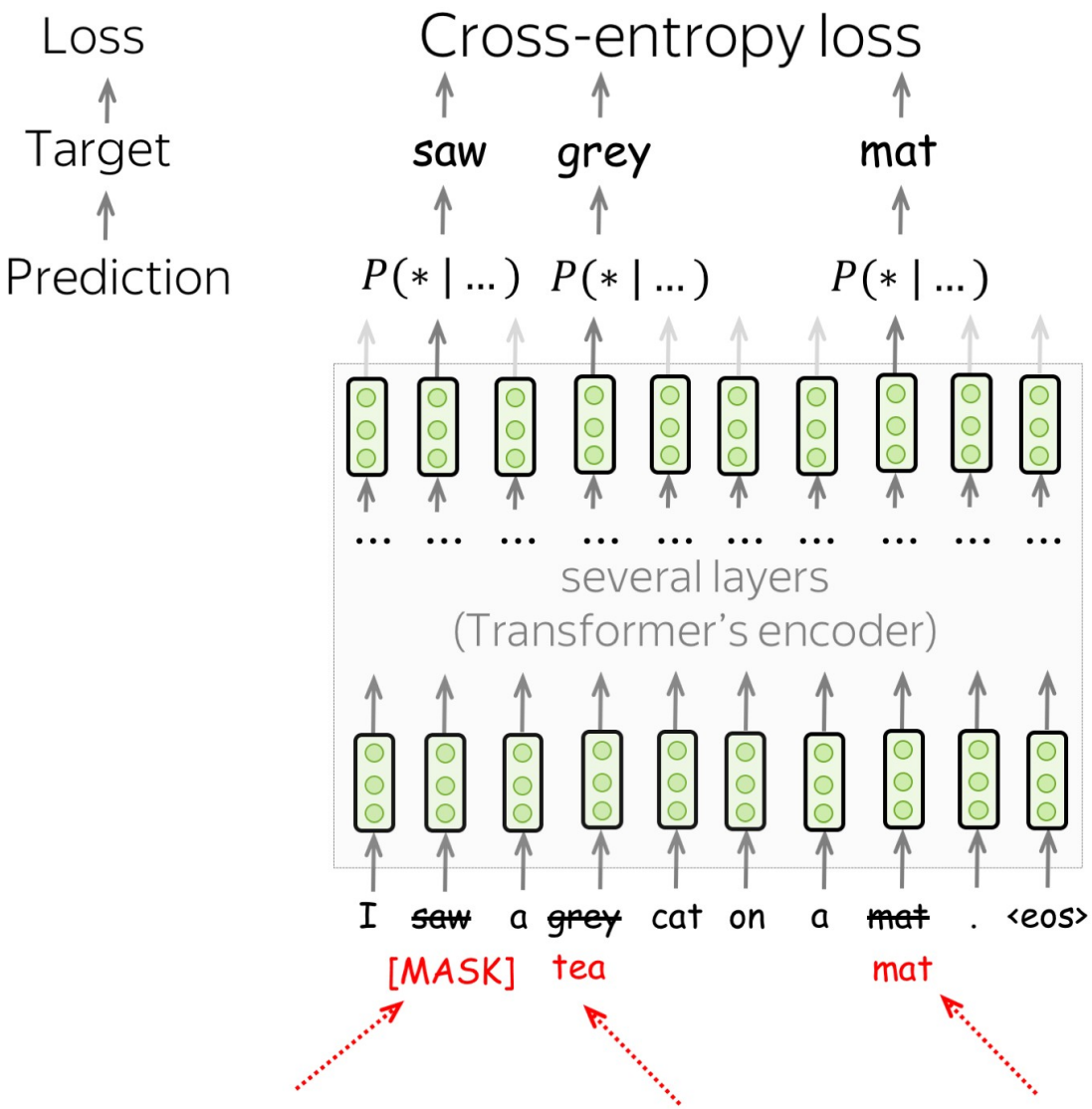
Decoder-encoder attention: target token looks at the source

queries – from decoder states; keys and values from encoder states

Decoder self-attention (masked): tokens look at the previous tokens

queries, keys, values are computed from decoder states

Recap: BERT



At each training step:

- pick randomly 15% of tokens
- replace each of the chosen tokens with something
- predict original chosen tokens

- **[MASK]**, with p = 80%
- Random token, with p = 10%
- Original token, with p = 10%

Recap: BERTology

What's the best model? https://super.gluebenchmark.com
I can't be bothered to check SuperGLUE T5 / deberta-v3
How do I get that model? https://huggingface.co
What is the best training objective? https://tinyurl.com/2bs8rdtt

What data do I use for pretraining?
Toy data for prototyping Wiki, OWT, BookCorpus
Large english datasets C4, PILE
Multilingual data: OSCAR, mC4, BigSci

How to prepare my own data? bigscience blog post

Recap: the humble language model

“Transformers make good language models”
everyone, 2017

“Language modeling kinda works for pretraining”
GPT-1 (2018), 117M weights, 5GB data

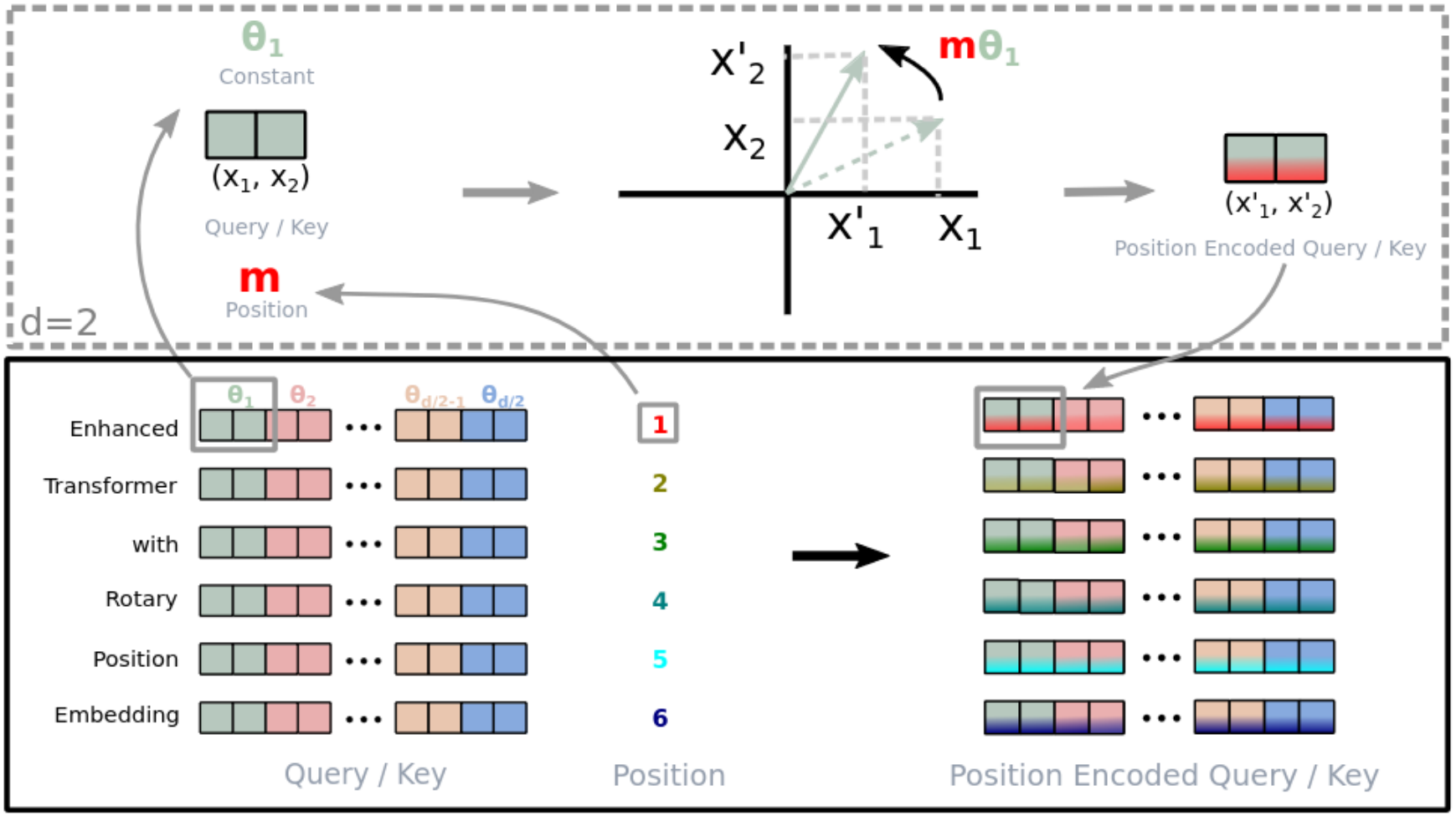
“Language models can do simple tasks without explicit training”
GPT-2 (2019), 1500M weights, 40GB data

What if we make it larger?
GPT-3 (2020), 175,000M weights, ~45,000 GB data

Recap: Rotary Embeddings

https://arxiv.org/abs/2104.09864
https://blog.eleuther.ai/rotary-embeddings/

A (bit overly complicated) explanation from the paper



Recap: MLP, MoE

https://arxiv.org/abs/2002.05202
https://arxiv.org/abs/1701.06538

Original FFN

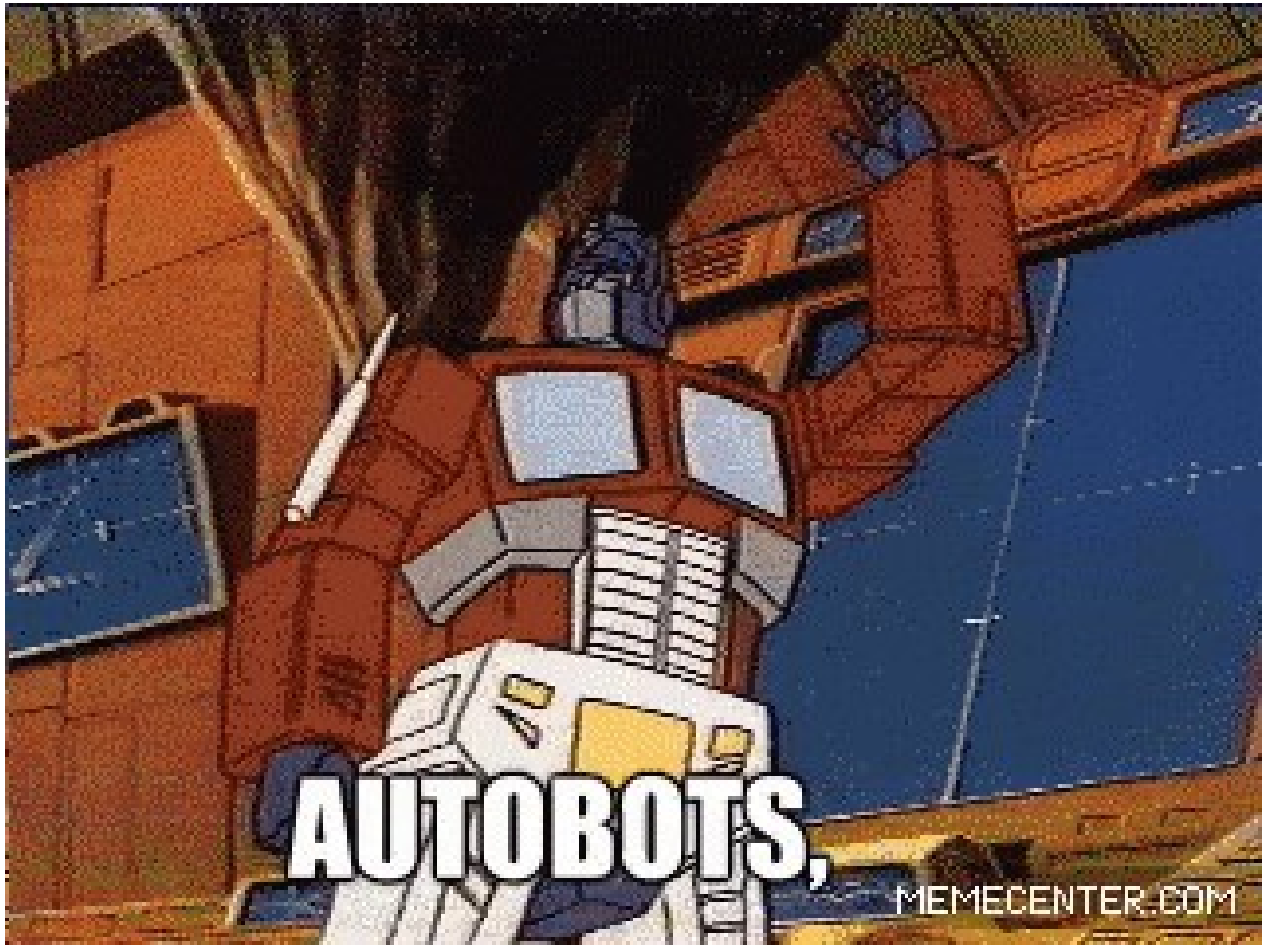
$$\mathrm{FFN}(x, W_1, W_2, b_1, b_2) = \max(0, xW_1 + b_1)W_2 + b_2$$

Gated FFN variants

$$\mathrm{FFN}_{\mathrm{GLU}}(x, W, V, W_2) = (\sigma(xW) \otimes xV)W_2$$
$$\mathrm{FFN}_{\mathrm{Bilinear}}(x, W, V, W_2) = (xW \otimes xV)W_2$$
$$\mathrm{FFN}_{\mathrm{ReGLU}}(x, W, V, W_2) = (\max(0, xW) \otimes xV)W_2$$
$$\mathrm{FFN}_{\mathrm{GEGLU}}(x, W, V, W_2) = (\mathrm{GELU}(xW) \otimes xV)W_2$$
$$\mathrm{FFN}_{\mathrm{SwiGLU}}(x, W, V, W_2) = (\mathrm{Swish}_1(xW) \otimes xV)W_2$$

MoE: choose one or few of K MLPs with a linear gate (trainable)
only run the chosen “experts”, the rest stay inactive.

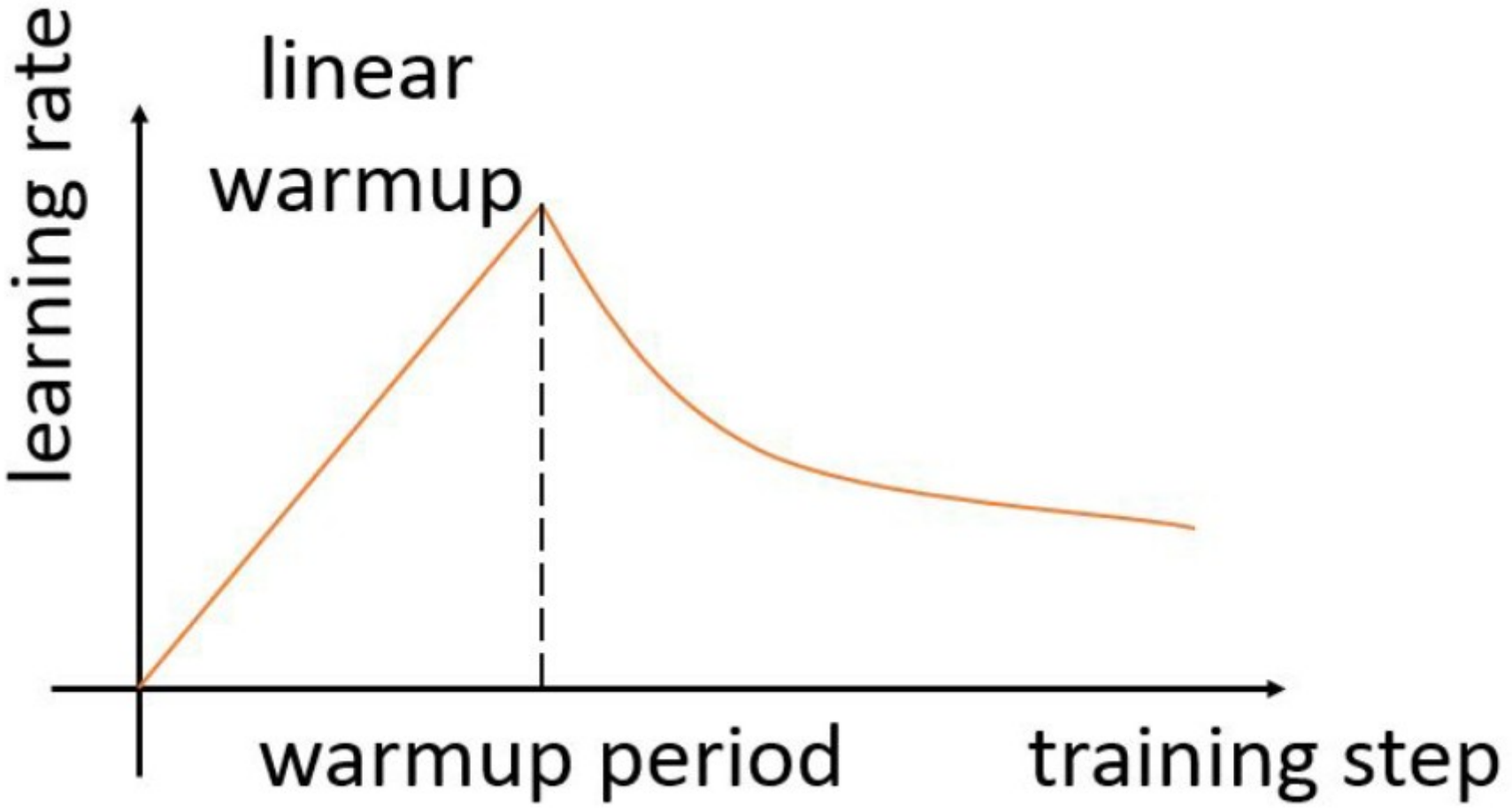
Part I: How to Train your Transformer



Training Tips for Transformers

https://arxiv.org/abs/1804.00247

Learning rate “warm-up”
from Vaswani et al. (2017)



Training Tips for Transformers

https://arxiv.org/abs/1804.00247

Learning rate “warm-up”
from Vaswani et al. (2017)

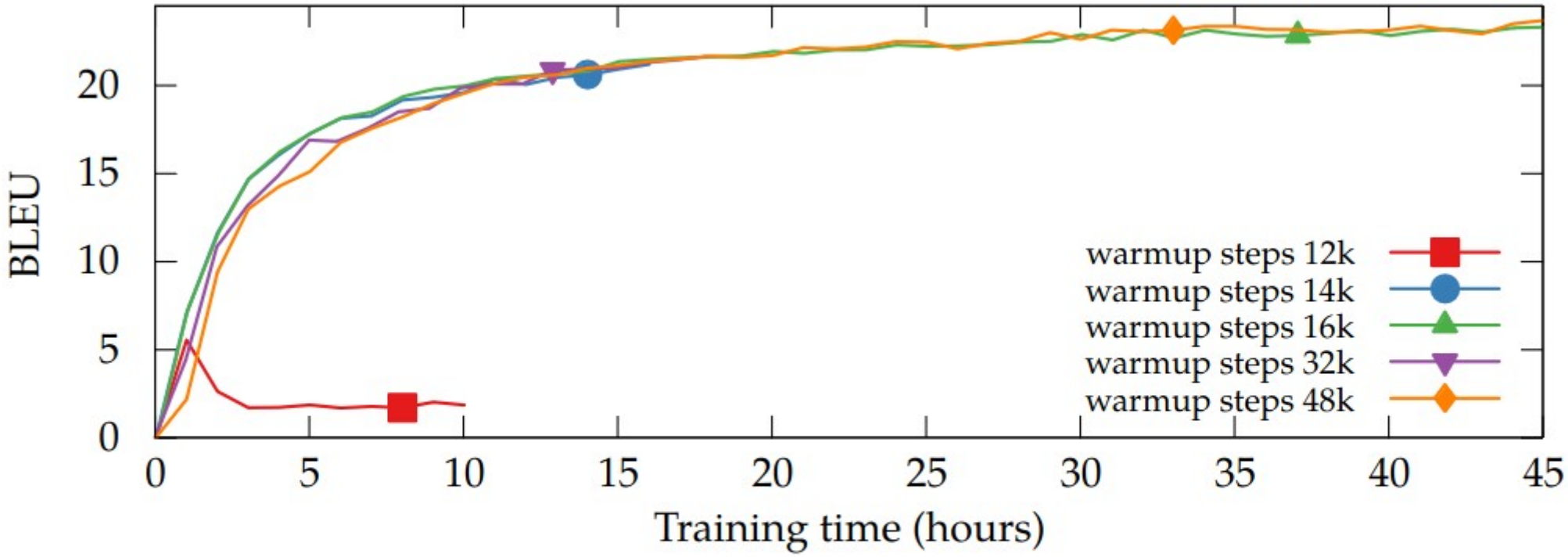


Figure 8: Effect of the warmup steps on a single GPU. All trained on CzEng 1.0 with the default batch size (1500) and learning rate (0.20).

Training Tips for Transformers

https://arxiv.org/abs/1804.00247

Batch needs to be large enough

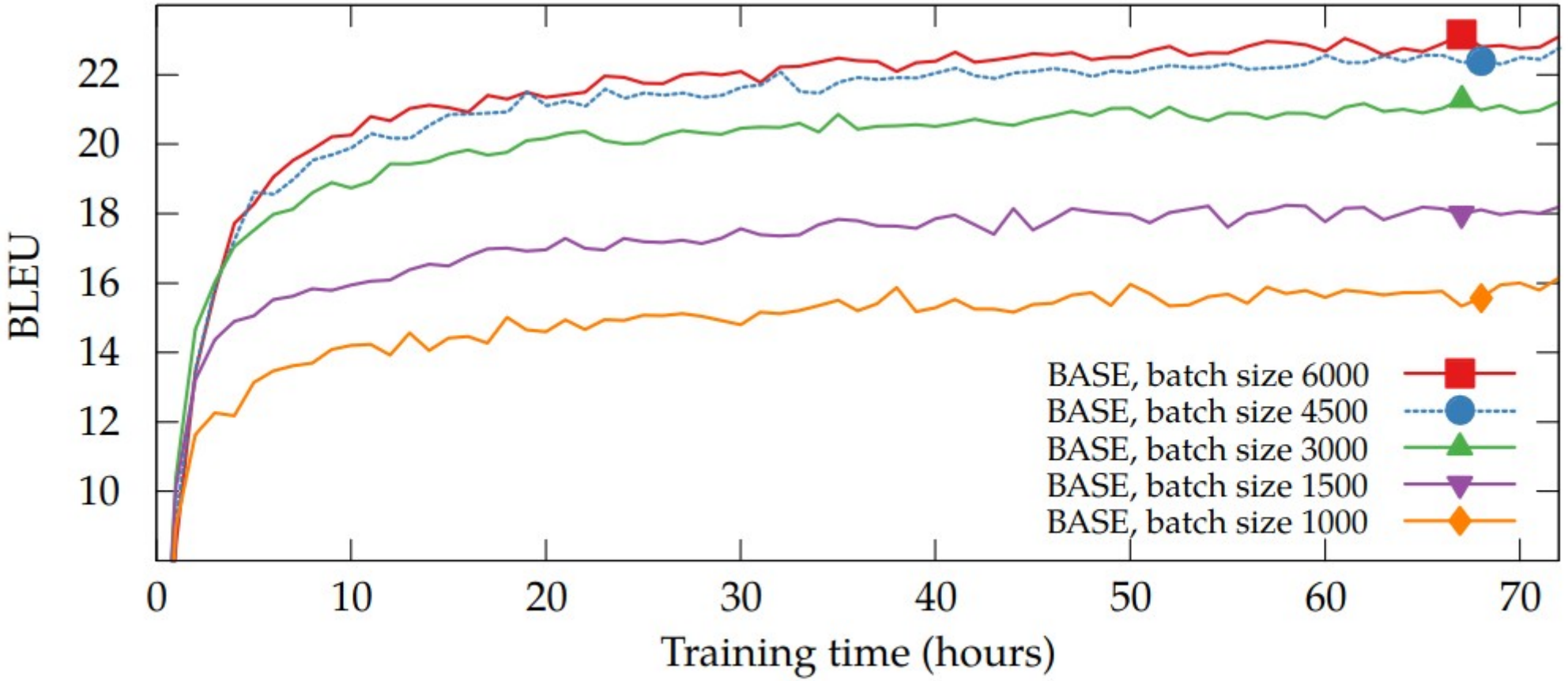


Figure 5: Effect of the batch size with the BASE model. All trained on a single GPU.

Training Tips for Transformers

https://arxiv.org/abs/1804.00247

Batch needs to be large enough

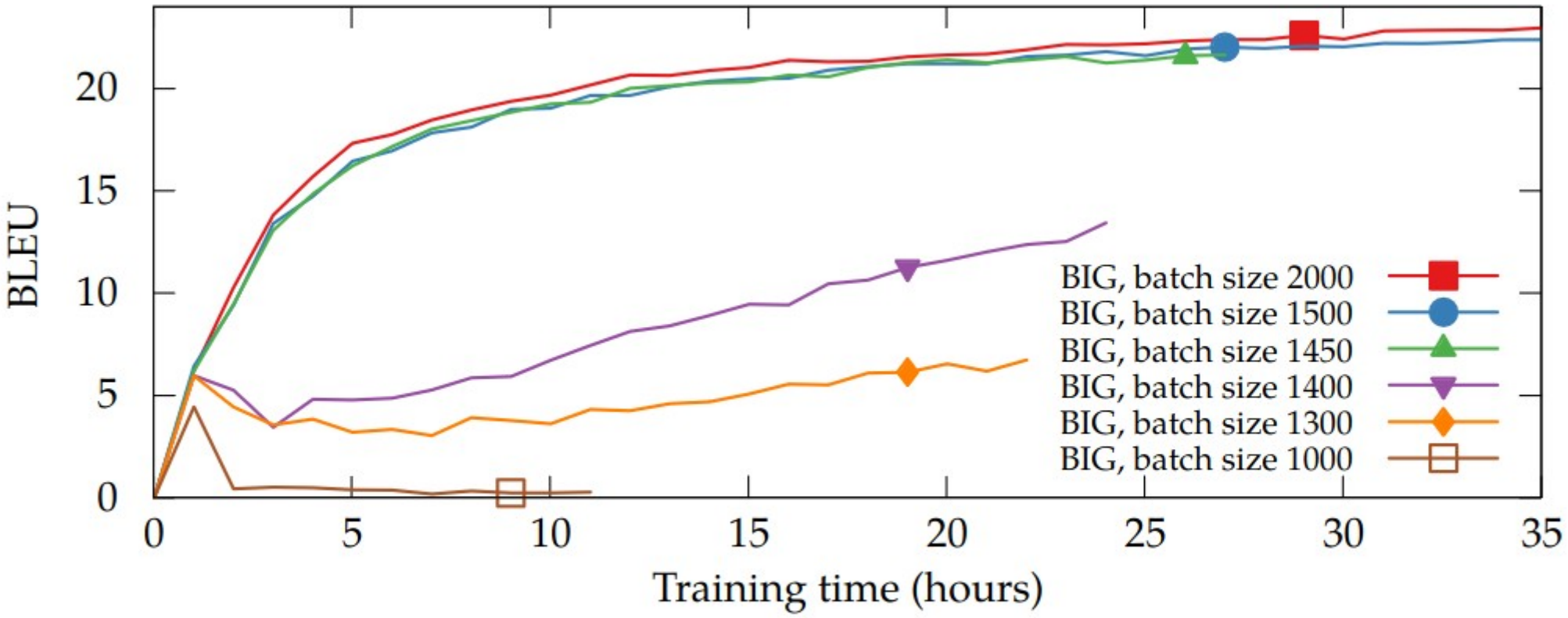


Figure 6: Effect of the batch size with the BIG model. All trained on a single GPU.

Training Tips for Transformers

https://arxiv.org/abs/1804.00247

From Vaswani et al (2017):

- Adam optimizer (SGD is way worse)
- Learning rate warmup / inverse sqrt decay
- Large batch sizes – accumulate over multiple fwd/bwd passes
- Form batches out of sentences of similar length
- Scale attention logits by 1 / sqrt(dim)

Other works (e.g. GPT-3)

- A ton of pre-processing tricks, e.g. concatenate texts
- Mixed precision training – Bfloat16 if able; float16 + hacks if not
- Adam-like optimizers: LAMB (large batch), AdaFactor(low memory) & many more
- Warm-up batch size – first few batches have fewer tokens
- Warm-up sequence length

Recap: Prompt Engineering

Playground

Load a preset...

Save

The following is a conversation of an alien coming in first contact with the human race. The Alien really enjoys vacations on Mars and the human it is talking to likes pizza.

A: Hello there! I'm an alien from a faraway planet. I'm here on vacation and I'm really enjoying myself. I love Mars and all the amazing things to see and do here.

B: Wow, that's amazing! I've always wanted to visit another planet. What's it like where you're from?

A: It's very different from here. Our planet is much larger and there are many more different kinds of creatures. We don't have any vacations, but we do have a lot of work.

B: That sounds pretty different. I'm glad you're enjoying your vacation here. Do you like pizza?

A: Yes, I love pizza! It's one of my favorite things to eat here on Earth.

Submit

194

Prompt Tuning

https://aclanthology.org/2021.emnlp-main.243.pdf

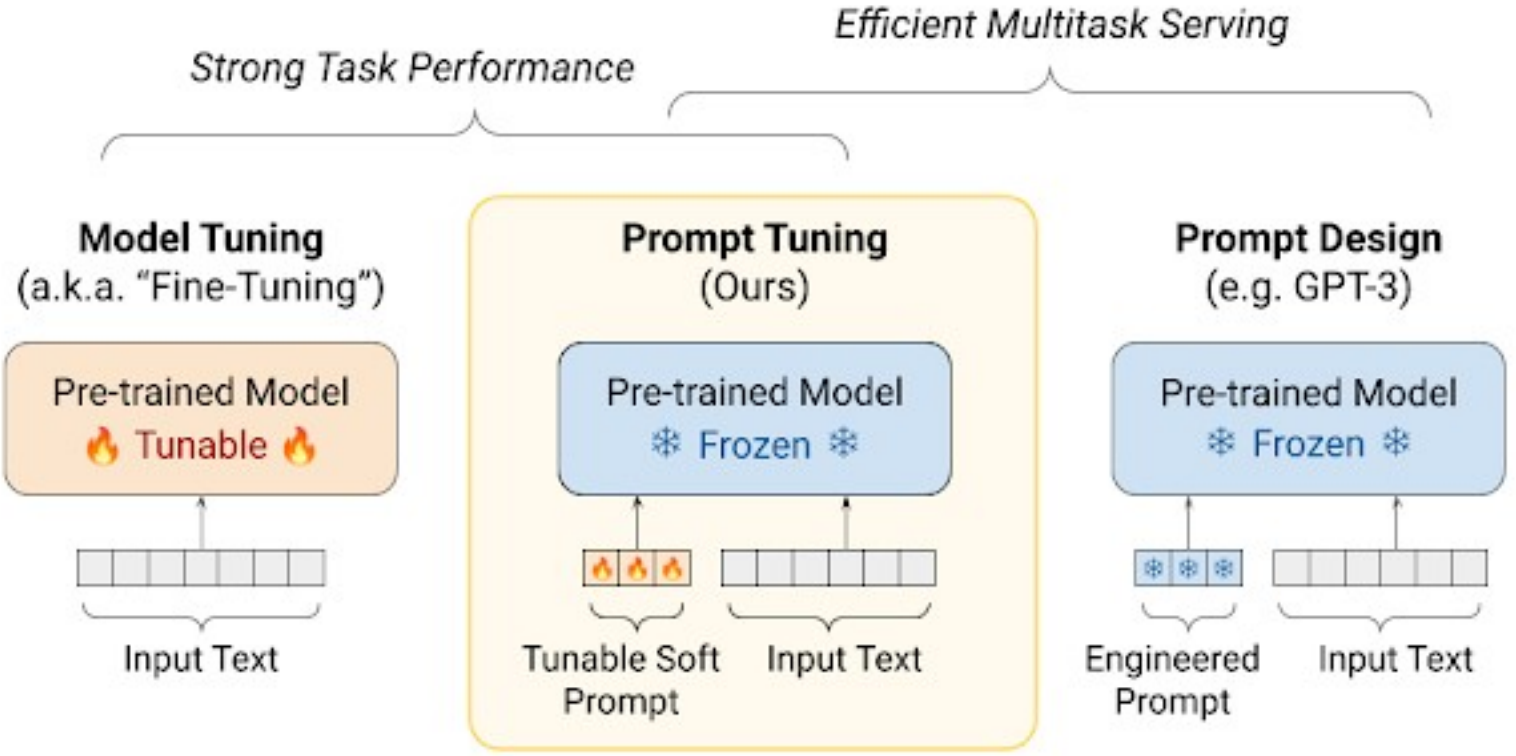
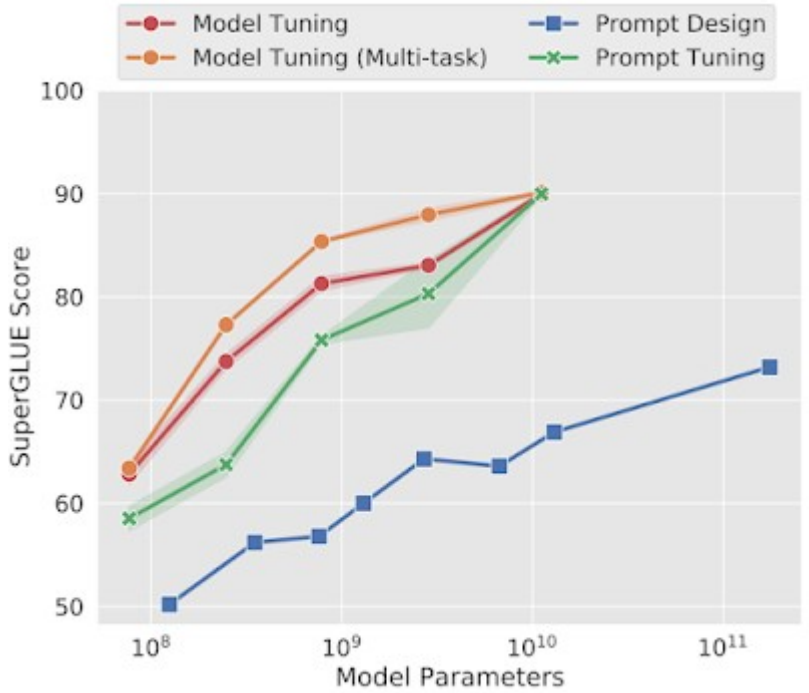


Image Credit: https://ai.googleblog.com/2022/02/guiding-frozen-language-models-with.html

Prompt Tuning

https://aclanthology.org/2021.emnlp-main.243.pdf

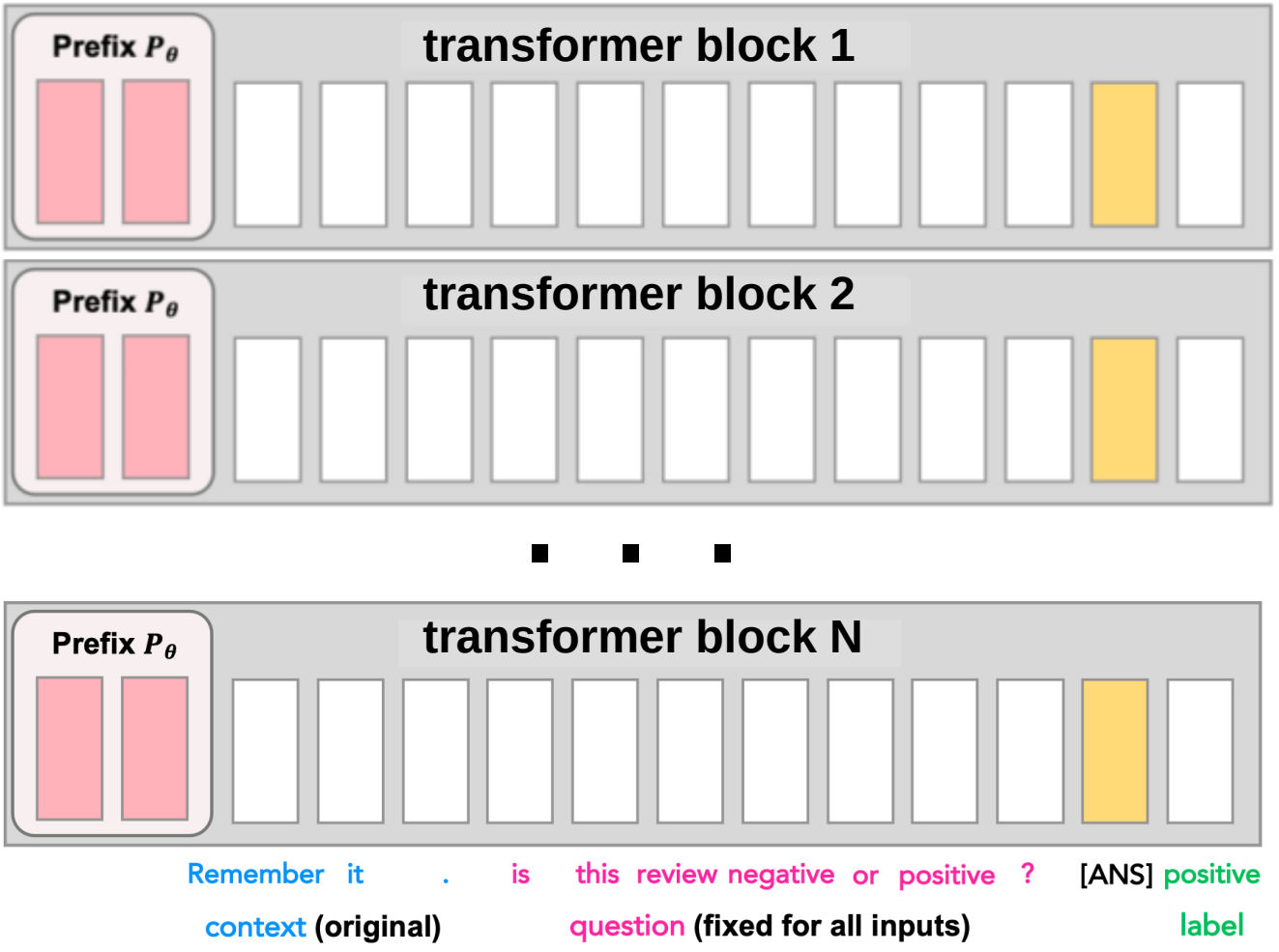
Prompt Tuning gets more competitive with scale!



Prefix tuning: p-tuning goes deep

https://arxiv.org/abs/2101.00190

https://arxiv.org/abs/2110.07602

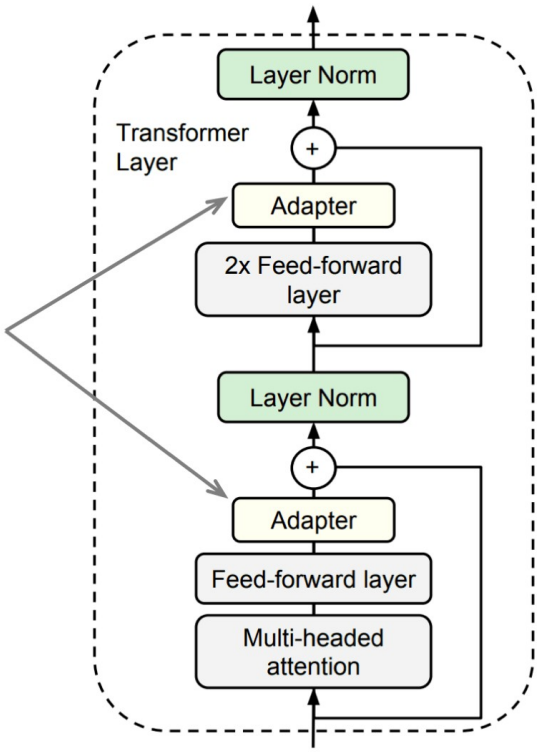


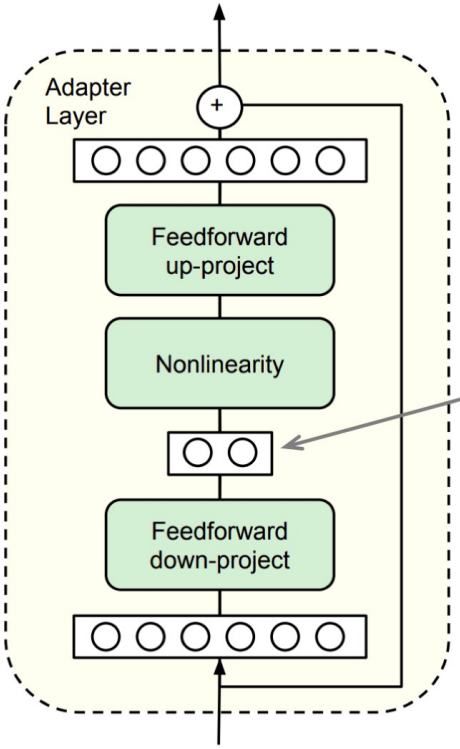
Adapters

https://arxiv.org/abs/1902.00751

Core idea: train small sub-networks

Only these are trained, everything else is fixed and is the same for all tasks





Small hidden size, i.e. an adaptor has only a few parameters (which is good!)

Adapters can do language adaptation

https://arxiv.org/abs/2204.04873

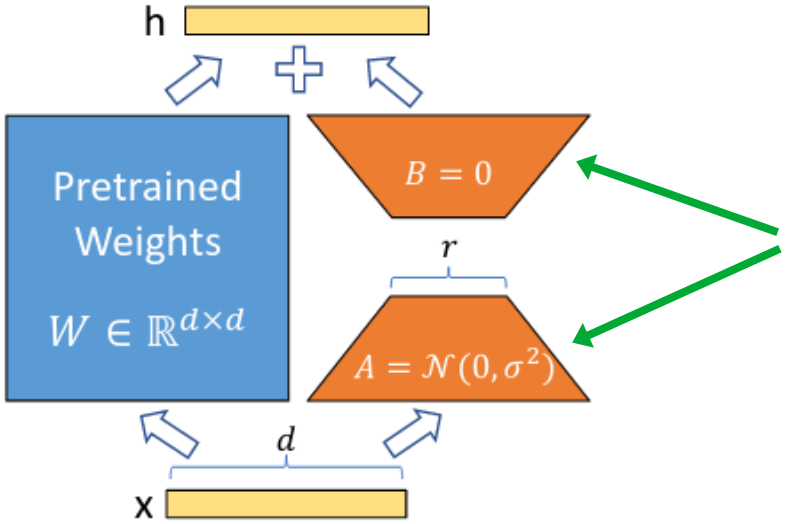
Generalize BLOOM to unseen languages without full model training
Only adapters and token embeddings are trained

	Models	Strategies	Ckpt.	Emb.	Adpt. Red.	(p.) de	en→ de	de→ de	(p.) ko	en→ ko	ko→ ko
(1)	mBERT_{BASE}	-	-	-	-	-	70.0	75.5	-	69.7	72.9
(2)	XLMR_{LARGE}	-	-	-	-	-	82.5	85.4	-	80.4	86.4
(3)	$\text{XGLM}_{1.7B}$	-	-	-	-	45.4	-	-	45.17	-	-
(4)	BigScience	-	-	-	-	34.1	44.8	67.4	-	-	-
(5)	BigScience	Emb	118,500	wte,wpe	-	41.4	50.7	74.3	34.4	45.6	53.4
(6)	BigScience	Emb→Adpt	118,500	wte,wpe	16	40.0	50.5	69.9	33.8	40.4	51.8
(7)	**BigScience**	**Emb+Adpt**	**118,500**	**wte**	**16**	**42.4**	**58.4**	**73.3**	**38.8**	**49.7**	**55.7**
(8)	BigScience	Emb+Adpt	118,500	wte	48	42.4	57.6	73.7	36.3	48.3	52.9
(9)	BigScience	Emb+Adpt	118,500	wte	384	42.4	55.3	74.2	37.5	49.4	54.6
(10)	BigScience	Emb+Adpt	100,500	wte	16	44.3	56.9	73.2	37.5	48.6	50.8
(11)	BigScience	Emb+Adpt	12,000	wte	16	33.5	55.2	70.5	32.9	46.4	53.3
(12)	BigScience	Emb+Adpt	100,500	wte,wpe	16	-	-	-	37.5	53.5	63.5
(13)	BigScience	Emb+Adpt	118,500	wte,wpe	16	44.7	64.9	73.0	-	-	-

LoRA

https://arxiv.org/abs/2106.09685

Add adapters in parallel with linear layers

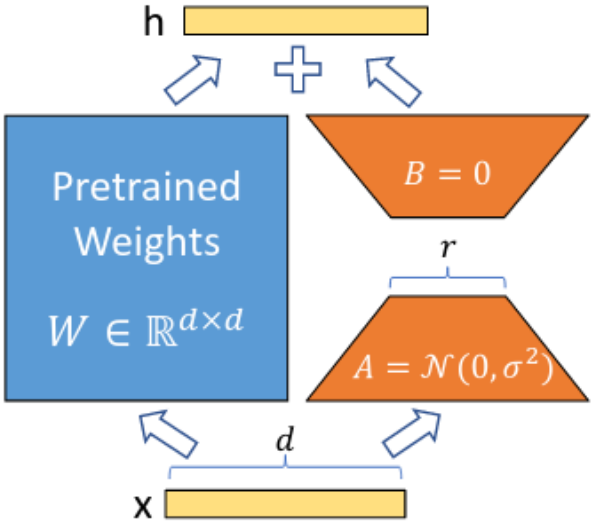


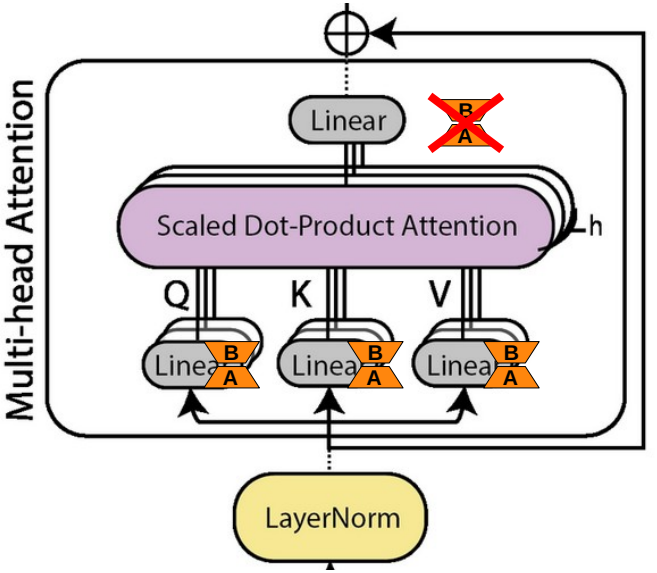
train only A & B

LoRA

https://arxiv.org/abs/2106.09685

Add adapters in parallel with linear layers





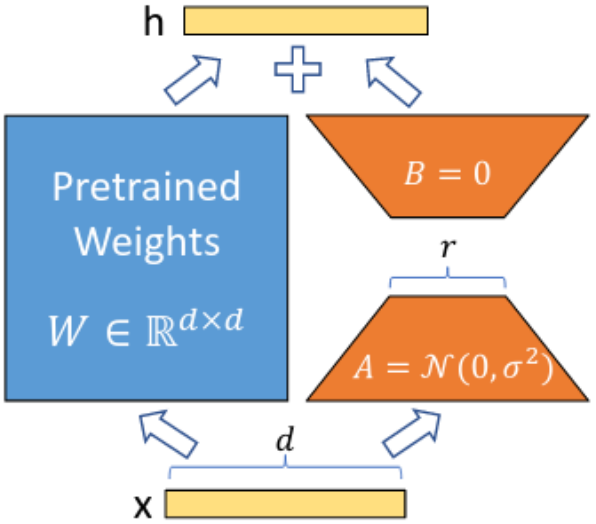
Add LoRA adapters to Q K & V layers
(but not to attention O, MLP, embeddings, logits)

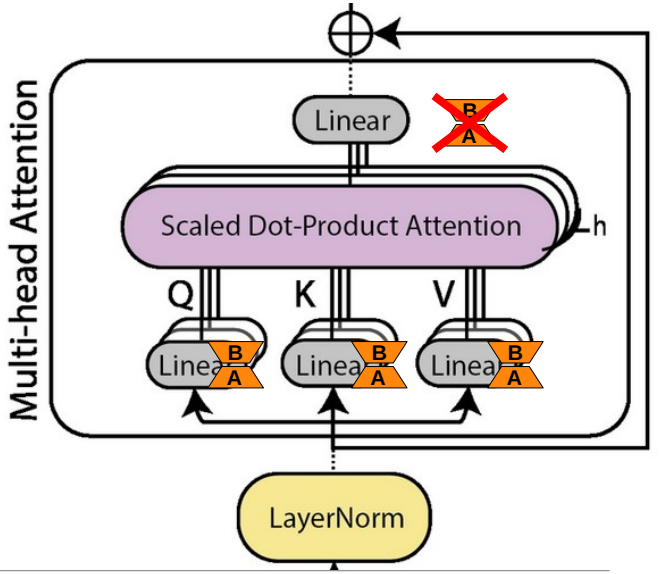
*Subsequent papers: add to QKV**O** and MLP*

LoRA

https://arxiv.org/abs/2106.09685

Add adapters in parallel with linear layers





Model&Method	# Trainable Parameters	WikiSQL Acc. (%)	MNLI-m Acc. (%)	SAMSum R1/R2/RL
GPT-3 (FT)	175,255.8M	**73.8**	89.5	52.0/28.0/44.5
GPT-3 (BitFit)	14.2M	71.3	91.0	51.3/27.4/43.5
GPT-3 (PreEmbed)	3.2M	63.1	88.6	48.3/24.2/40.5
GPT-3 (PreLayer)	20.2M	70.1	89.5	50.8/27.3/43.5
GPT-3 ($\text{Adapter}^{\text{H}}$)	7.1M	71.9	89.8	53.0/28.9/44.8
GPT-3 ($\text{Adapter}^{\text{H}}$)	40.1M	73.2	**91.5**	53.2/29.0/45.1
GPT-3 (LoRA)	4.7M	73.4	**91.7**	**53.8/29.8/45.9**
GPT-3 (LoRA)	37.7M	**74.0**	**91.6**	53.4/29.2/45.1

LoRA

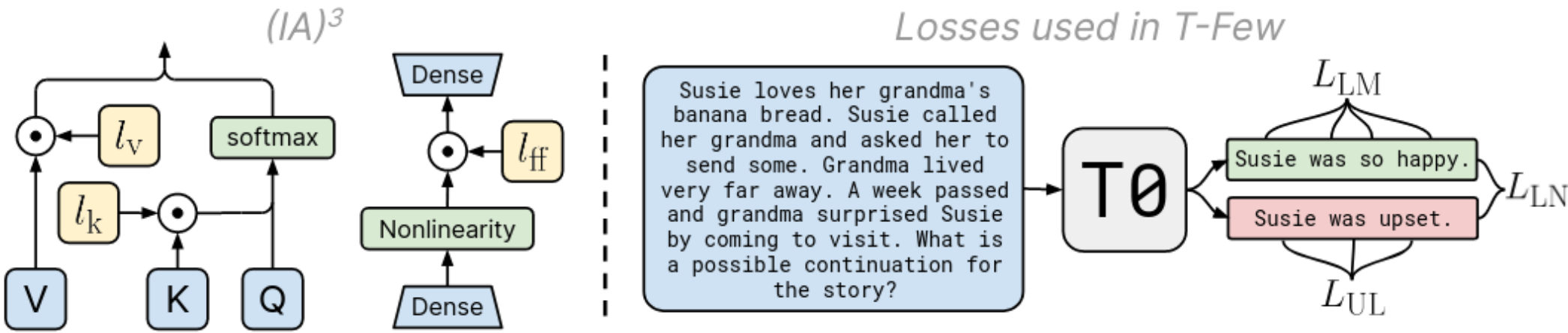
https://arxiv.org/abs/2106.09685

Below: RoBERTa-base/large & DeBerta XXL

Model & Method	# Trainable Parameters	MNLI	SST-2	MRPC	CoLA	QNLI	QQP	RTE	STS-B	Avg.
RoB_{base} (FT)*	125.0M	**87.6**	94.8	90.2	**63.6**	92.8	**91.9**	78.7	91.2	86.4
RoB_{base} (BitFit)*	0.1M	84.7	93.7	**92.7**	62.0	91.8	84.0	81.5	90.8	85.2
RoB_{base} (Adpt^{D})*	0.3M	$87.1_{\pm.0}$	$94.2_{\pm.1}$	$88.5_{\pm1.1}$	$60.8_{\pm.4}$	$93.1_{\pm.1}$	$90.2_{\pm.0}$	$71.5_{\pm2.7}$	$89.7_{\pm.3}$	84.4
RoB_{base} (Adpt^{D})*	0.9M	$87.3_{\pm.1}$	$94.7_{\pm.3}$	$88.4_{\pm.1}$	$62.6_{\pm.9}$	$93.0_{\pm.2}$	$90.6_{\pm.0}$	$75.9_{\pm2.2}$	$90.3_{\pm.1}$	85.4
RoB_{base} (LoRA)	0.3M	$87.5_{\pm.3}$	$\mathbf{95.1}_{\pm.2}$	$89.7_{\pm.7}$	$63.4_{\pm1.2}$	$\mathbf{93.3}_{\pm.3}$	$90.8_{\pm.1}$	$\mathbf{86.6}_{\pm.7}$	$\mathbf{91.5}_{\pm.2}$	**87.2**
$\text{RoB}_{\text{large}}$ (FT)*	355.0M	90.2	**96.4**	**90.9**	68.0	94.7	**92.2**	86.6	92.4	88.9
$\text{RoB}_{\text{large}}$ (LoRA)	0.8M	$\mathbf{90.6}_{\pm.2}$	$96.2_{\pm.5}$	$\mathbf{90.9}_{\pm1.2}$	$\mathbf{68.2}_{\pm1.9}$	$\mathbf{94.9}_{\pm.3}$	$91.6_{\pm.1}$	$\mathbf{87.4}_{\pm2.5}$	$\mathbf{92.6}_{\pm.2}$	**89.0**
$\text{RoB}_{\text{large}}$ (Adpt^{P})†	3.0M	$90.2_{\pm.3}$	$96.1_{\pm.3}$	$90.2_{\pm.7}$	$\mathbf{68.3}_{\pm1.0}$	$\mathbf{94.8}_{\pm.2}$	$\mathbf{91.9}_{\pm.1}$	$83.8_{\pm2.9}$	$92.1_{\pm.7}$	88.4
$\text{RoB}_{\text{large}}$ (Adpt^{P})†	0.8M	$\mathbf{90.5}_{\pm.3}$	$\mathbf{96.6}_{\pm.2}$	$89.7_{\pm1.2}$	$67.8_{\pm2.5}$	$\mathbf{94.8}_{\pm.3}$	$91.7_{\pm.2}$	$80.1_{\pm2.9}$	$91.9_{\pm.4}$	87.9
$\text{RoB}_{\text{large}}$ (Adpt^{H})†	6.0M	$89.9_{\pm.5}$	$96.2_{\pm.3}$	$88.7_{\pm2.9}$	$66.5_{\pm4.4}$	$94.7_{\pm.2}$	$92.1_{\pm.1}$	$83.4_{\pm1.1}$	$91.0_{\pm1.7}$	87.8
$\text{RoB}_{\text{large}}$ (Adpt^{H})†	0.8M	$90.3_{\pm.3}$	$96.3_{\pm.5}$	$87.7_{\pm1.7}$	$66.3_{\pm2.0}$	$94.7_{\pm.2}$	$91.5_{\pm.1}$	$72.9_{\pm2.9}$	$91.5_{\pm.5}$	86.4
$\text{RoB}_{\text{large}}$ (LoRA)†	0.8M	$\mathbf{90.6}_{\pm.2}$	$96.2_{\pm.5}$	$\mathbf{90.2}_{\pm1.0}$	$68.2_{\pm1.9}$	$\mathbf{94.8}_{\pm.3}$	$91.6_{\pm.2}$	$\mathbf{85.2}_{\pm1.1}$	$\mathbf{92.3}_{\pm.5}$	**88.6**
DeB_{XXL} (FT)*	1500.0M	91.8	**97.2**	92.0	72.0	**96.0**	92.7	93.9	92.9	91.1
DeB_{XXL} (LoRA)	4.7M	$\mathbf{91.9}_{\pm.2}$	$96.9_{\pm.2}$	$\mathbf{92.6}_{\pm.6}$	$\mathbf{72.4}_{\pm1.1}$	$\mathbf{96.0}_{\pm.1}$	$\mathbf{92.9}_{\pm.1}$	$\mathbf{94.9}_{\pm.4}$	$\mathbf{93.0}_{\pm.2}$	**91.3**

T-Few (IA3)

https://arxiv.org/abs/2205.05638

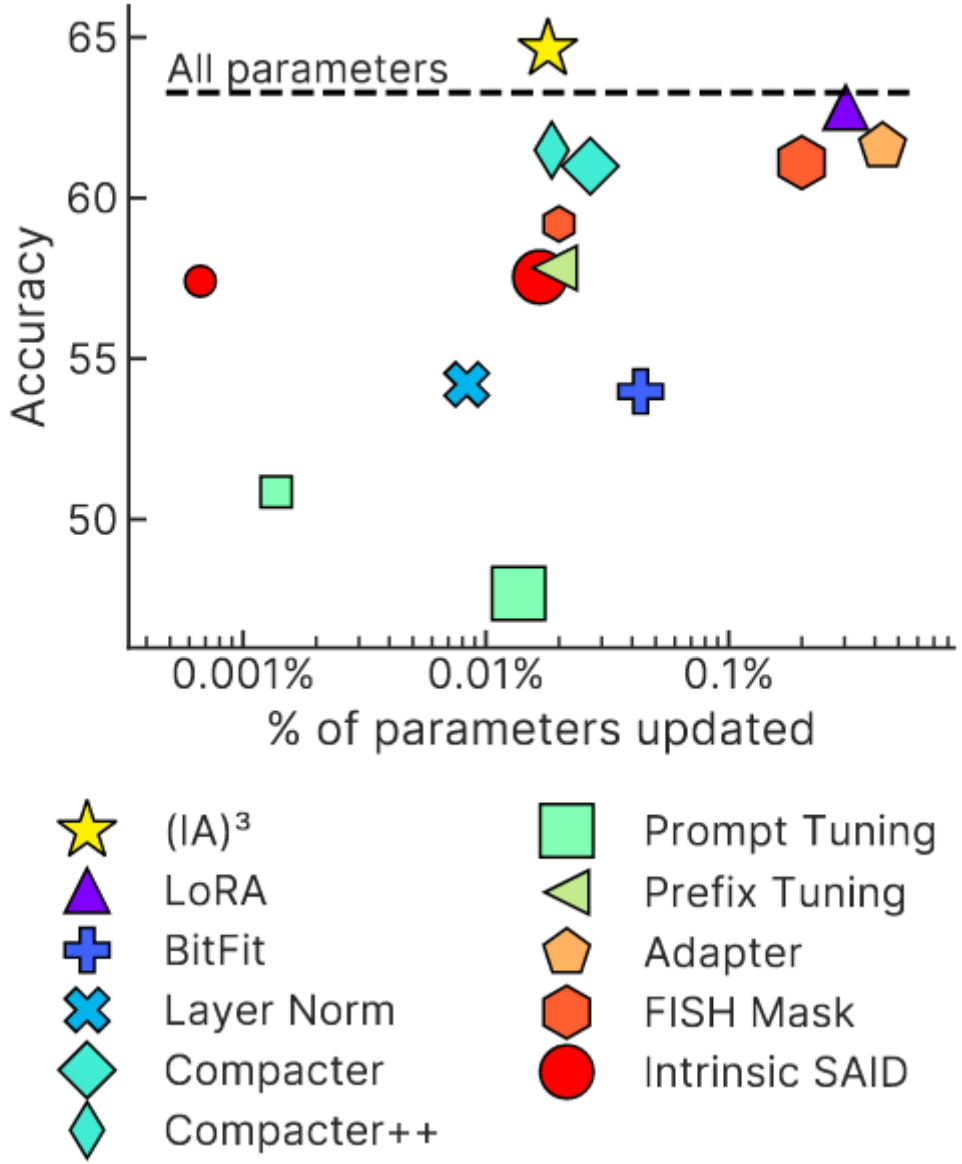


Accuracy and costs on held-out T0 tasks

Method	Inference FLOPs	Training FLOPs	Disk space	Acc.
T-Few	1.1e12	2.7e16	4.2 MB	72.4%
T0 [1]	1.1e12	0	0 B	66.9%
T5+LM [14]	4.5e13	0	16 kB	49.6%
GPT-3 6.7B [4]	5.4e13	0	16 kB	57.2%
GPT-3 13B [4]	1.0e14	0	16 kB	60.3%
GPT-3 175B [4]	1.4e15	0	16 kB	66.6%

T-Few (IA3)

https://arxiv.org/abs/2205.05638



TL;DR PEFT

***P**arameter **E**fficient **F**ine-**T**uning*

“When do I use this?”

10s-100s of examples = prompt engineering, IA3

100s, 1000s of examples = prompt tuning, IA3

more examples = LoRA adapters

“Which method do I use?”

Try latest p-tuning and latest adapters, compare

“Where can I play with these methods?”

https://huggingface.co/docs/peft/index

TL;DR PEFT

***P**arameter **E**fficient **F**ine-**T**uning*

That's just a rule of thumb!

~~**"When do I use this?"**~~

10s-100s of examples = prompt engineering, IA3
100s, 1000s of examples = prompt tuning, IA3
more examples = LoRA adapters

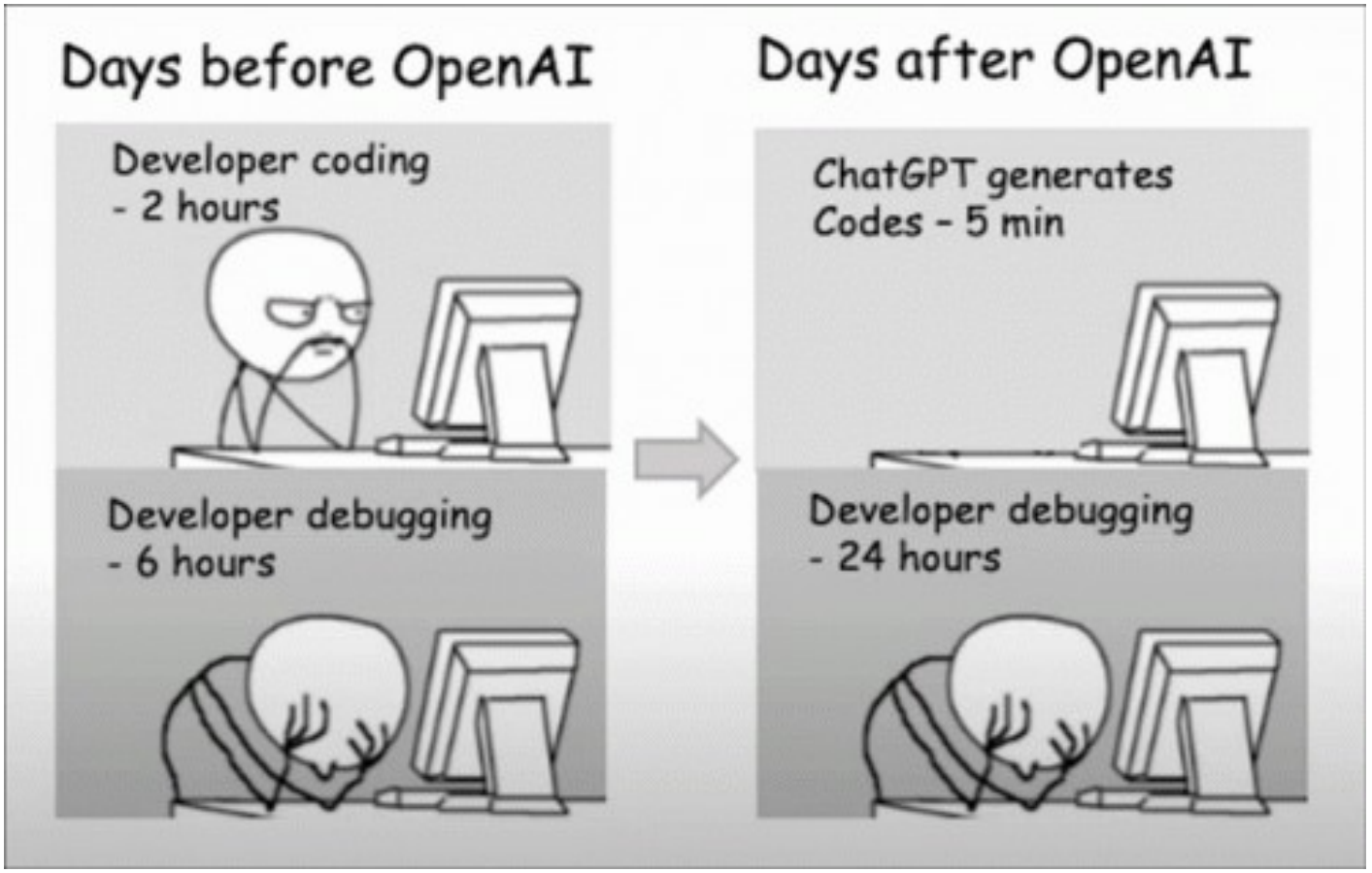
"Which method do I use?"

Try latest p-tuning and latest adapters, compare

"Where can I play with these methods?"

https://huggingface.co/docs/peft/index

Part II: LLM Finetuning Cycle
sft, rhlf, etc



What we want: Instruction Following

Paper: https://arxiv.org/abs/2203.02155

Prompt:
What is the purpose of the list C in the code below?

```
def binomial_coefficient(n, r):
    C = [0 for i in range(r + 1)];
    C[0] = 1;
    for i in range(1, n + 1):
        j = min(i, r);
        while j > 0:
            C[j] += C[j - 1];
            j -= 1;
    return C[r]
```

GPT-3 175B completion:
A. to store the value of C[0]
B. to store the value of C[1]
C. to store the value of C[i]
D. to store the value of C[i - 1]

InstructGPT 175B completion:
The list C in this code is used to store the values of the binomial coefficient as the function iterates through the values of n and r. It is used to calculate the value of the binomial coefficient for a given value of n and r, and stores the result in the final return value of the function.

Instruction Tuning

Training objective:

what we **want** vs
what we **told** model to do

What we **told** model to do:

- predict the next token on a webpage from the internet

Alignment →

What we want model to do:

- follow the user's instructions helpfully and safely

The language modeling objective is misaligned

Source: Training language models to follow instructions with human feedback, NeurIPS 2022

InstructGPT: basically ChatGPT

Paper: https://arxiv.org/abs/2203.02155

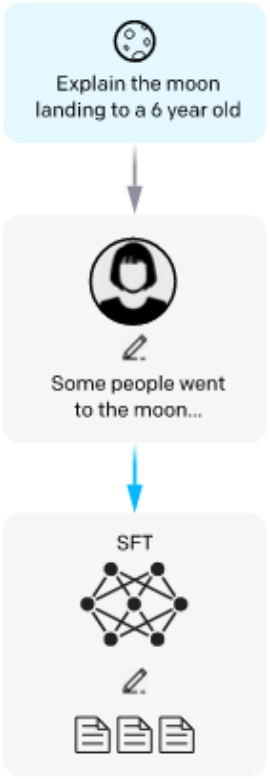
Step 1

Collect demonstration data, and train a supervised policy.

A prompt is sampled from our prompt dataset.

A labeler demonstrates the desired output behavior.

This data is used to fine-tune GPT-3 with supervised learning.



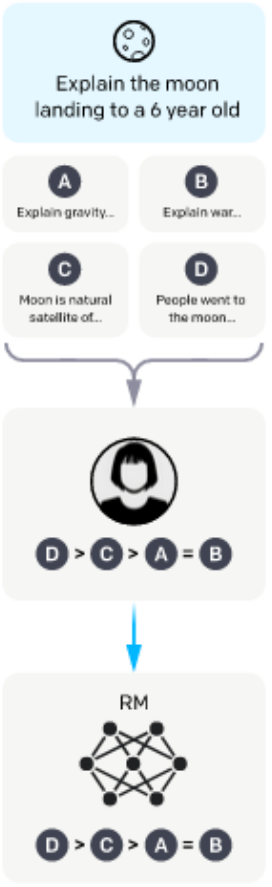
Step 2

Collect comparison data, and train a reward model.

A prompt and several model outputs are sampled.

A labeler ranks the outputs from best to worst.

This data is used to train our reward model.



Step 3

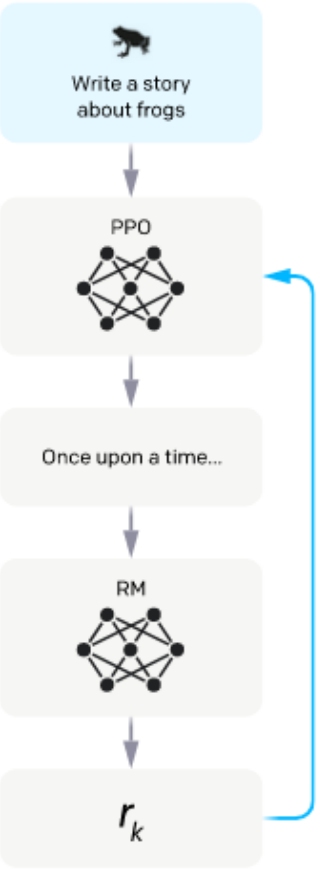
Optimize a policy against the reward model using reinforcement learning.

A new prompt is sampled from the dataset.

The policy generates an output.

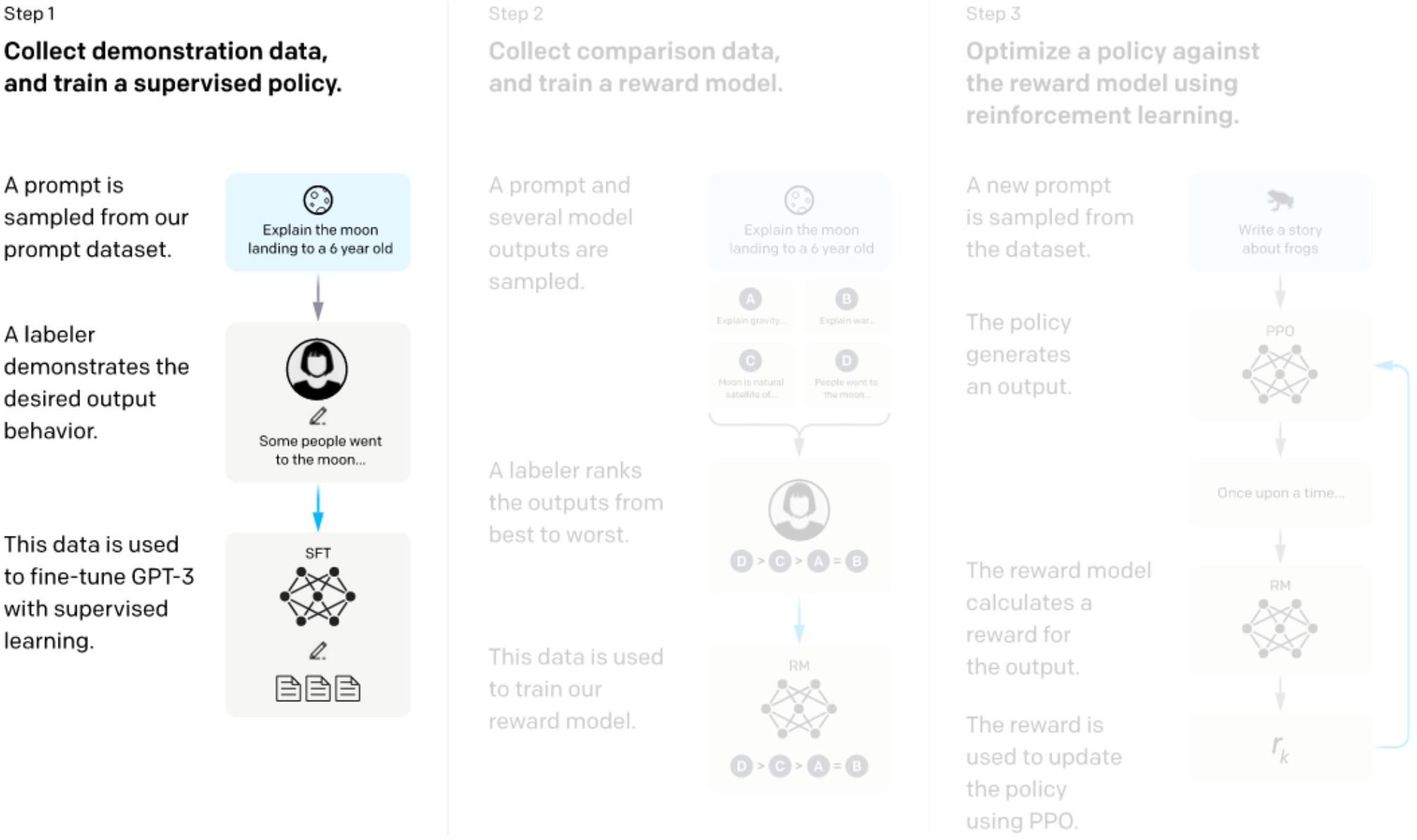
The reward model calculates a reward for the output.

The reward is used to update the policy using PPO.



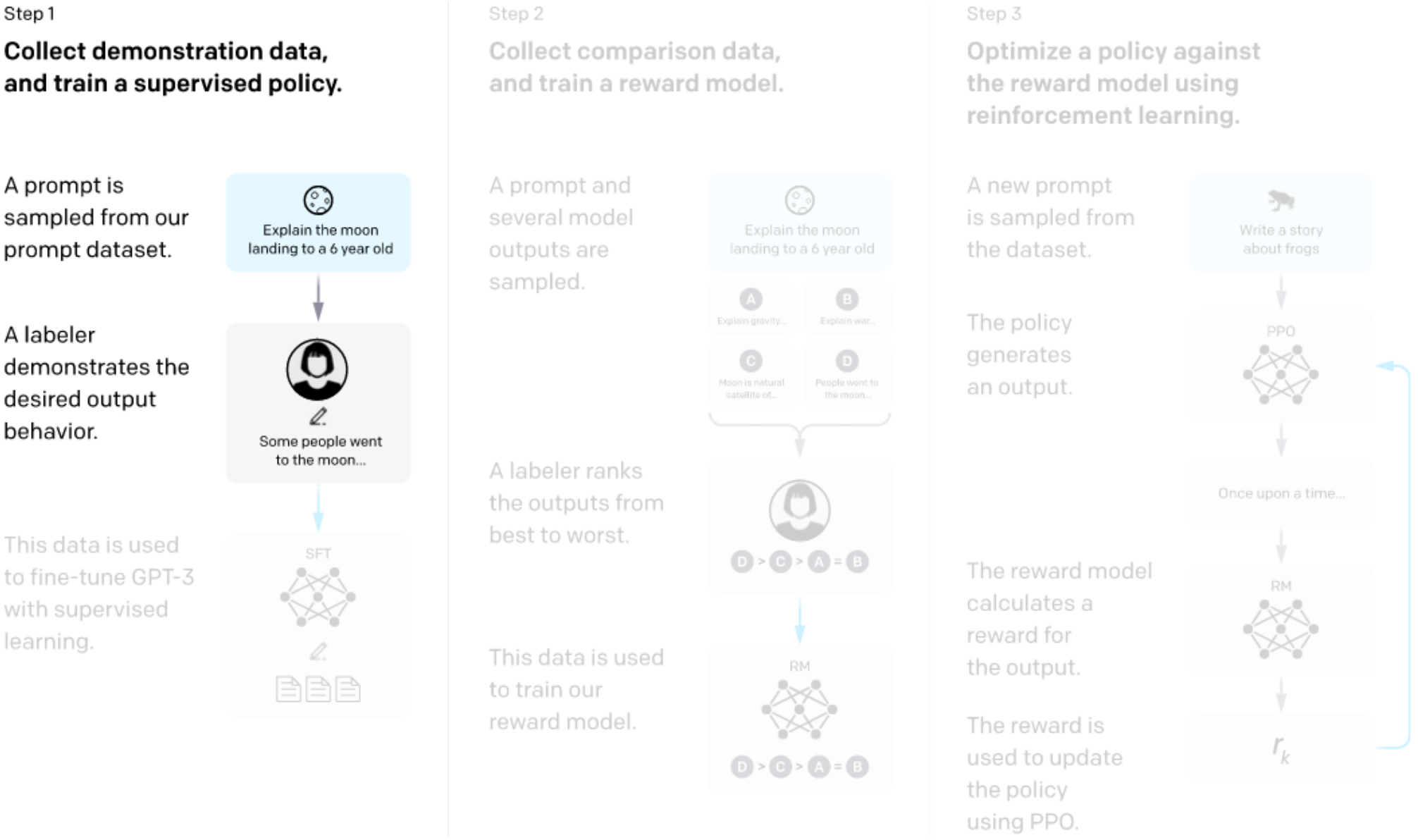
Stage 1: Supervised Fine-tuning

Paper: https://arxiv.org/abs/2203.02155



Stage 1: Supervised Fine-tuning

Paper: https://arxiv.org/abs/2203.02155



Stage 1: Supervised Fine-tuning

Paper: https://arxiv.org/abs/2203.02155

Prompt Dataset

Initial stage

Manually written by labelers

- Plain: come up with an arbitrary task, while ensuring the tasks had sufficient diversity
- Few-shot: come up with an instruction, and multiple query/response pairs for that instruction
- User-based: come up with prompts corresponding to some of the use-cases stated in waitlist applications to the OpenAI API

Later stage

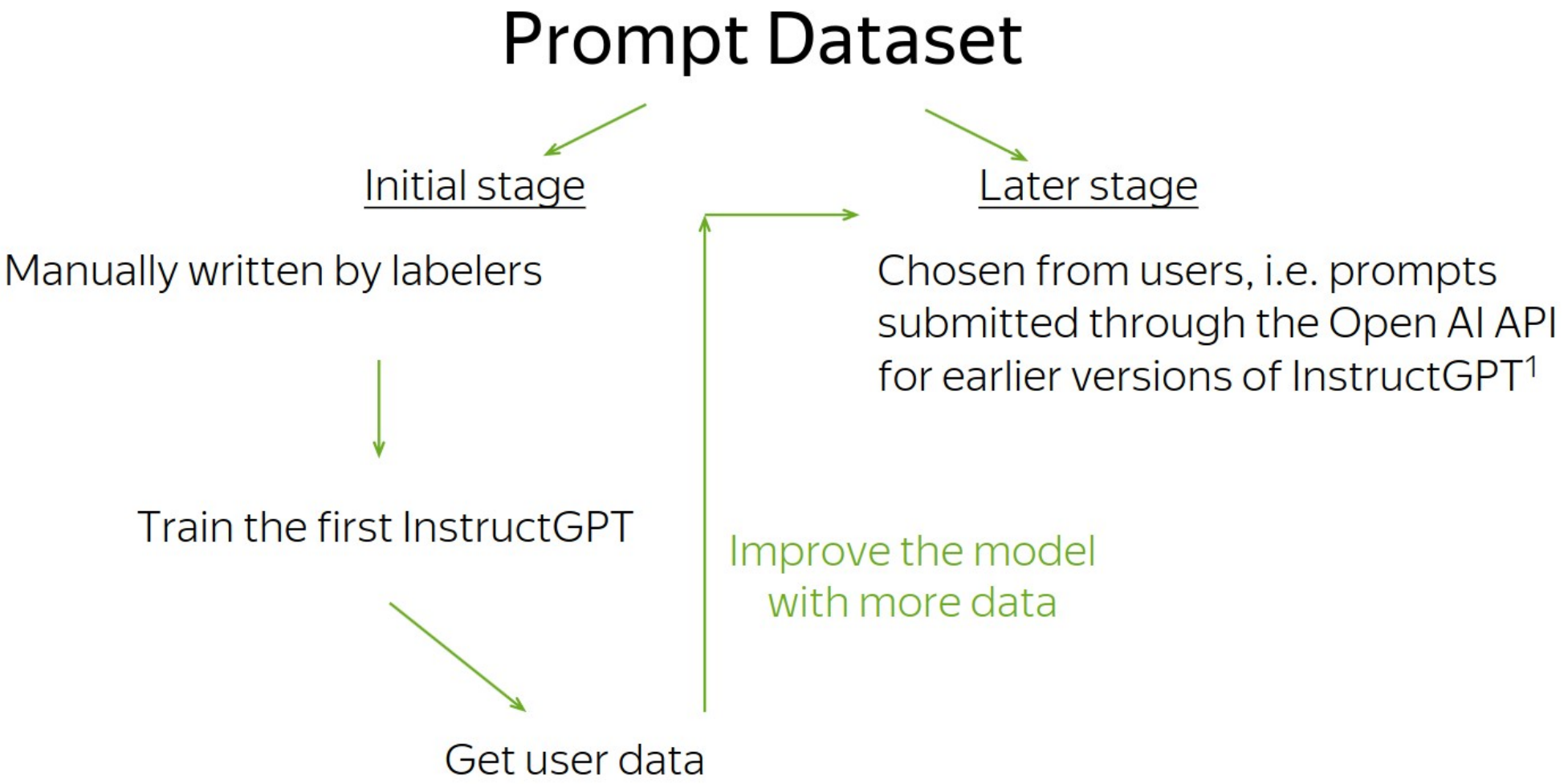
Chosen from users, i.e. prompts submitted through the Open AI API for earlier versions of InstructGPT[1]

- filter all prompts in the training split for personally identifiable information (PII)
- heuristically deduplicate prompts
- no more than 200 prompts per user ID
- create train, validation, and test splits based on user ID

[1]https://beta. openai.com/playground

Stage 1: Supervised Fine-tuning

Paper: https://arxiv.org/abs/2203.02155



[1]https://beta. openai.com/playground

Stage 1: Supervised Fine-tuning

Paper: https://arxiv.org/abs/2203.02155

Use cases

Use-case	(%)
Generation	45.6%
Open QA	12.4%
Brainstorming	11.2%
Chat	8.4%
Rewrite	6.6%
Summarization	4.2%
Classification	3.5%
Other	3.5%
Closed QA	2.6%
Extract	1.9%

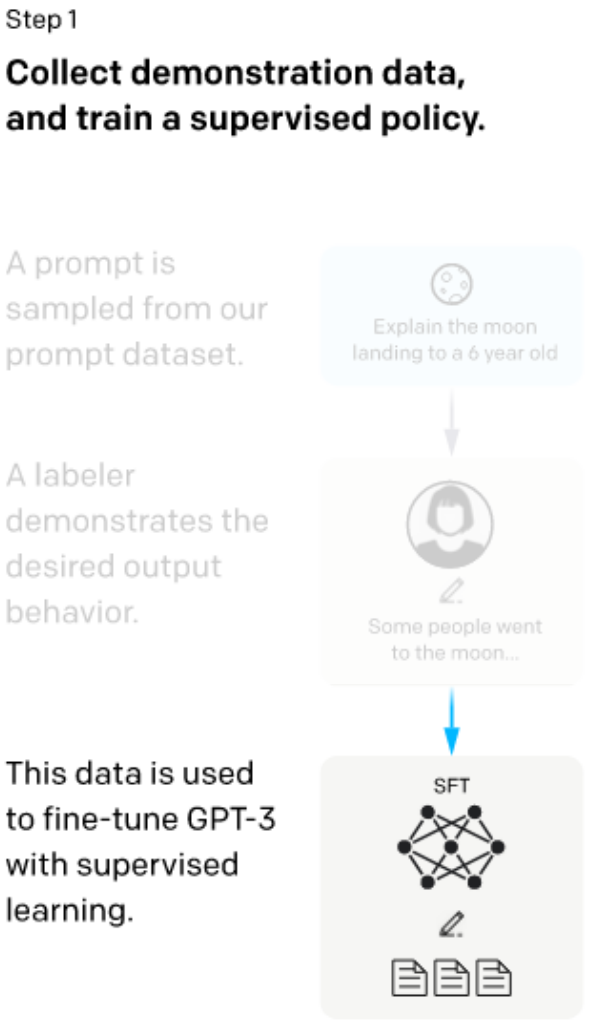
Examples

Use-case	Prompt
Brainstorming	List five ideas for how to regain enthusiasm for my career
Generation	Write a short story where a bear goes to the beach, makes friends with a seal, and then returns home.
Rewrite	This is the summary of a Broadway play: """ {summary} """ This is the outline of the commercial for that play: """

Source: Training language models to follow instructions with human feedback, NeurIPS 2022

Stage 1: Supervised Fine-tuning

Paper: https://arxiv.org/abs/2203.02155

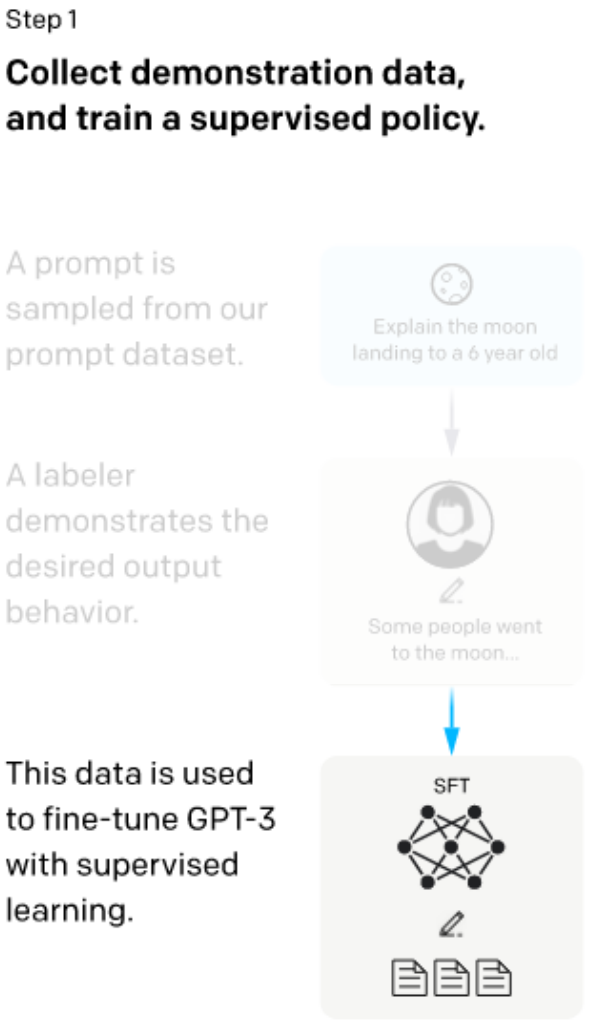


Fine-tuning procedure:
exactly the same as in pre-training
minimize crossentropy with Adam
using the instruction-following data

Cheaper version: train LoRA adapters
https://arxiv.org/abs/2305.14314

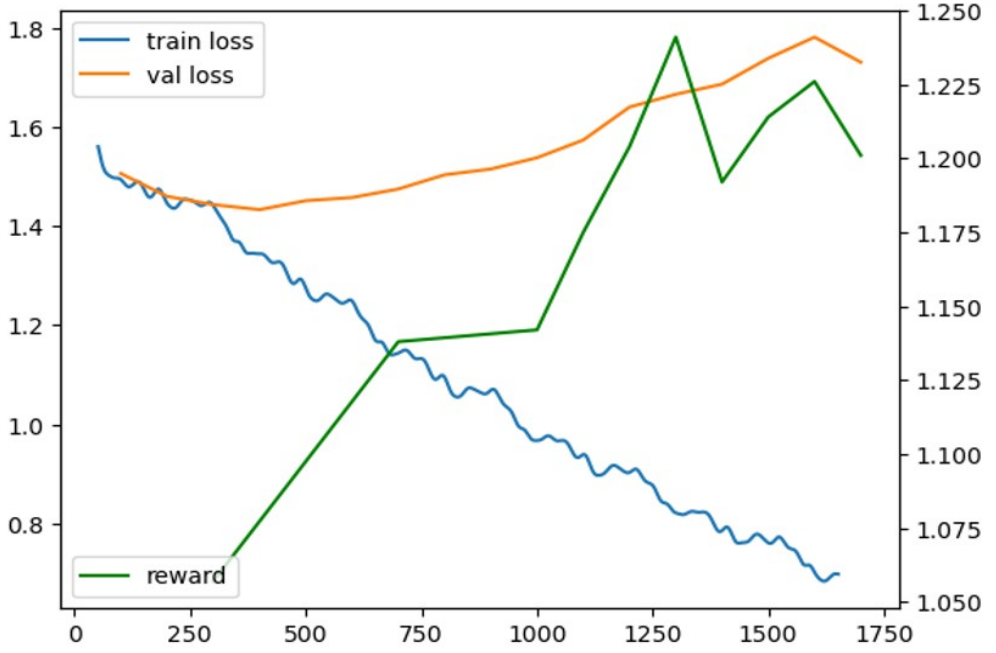
Stage 1: Supervised Fine-tuning

Paper: https://arxiv.org/abs/2203.02155



Fine-tuning procedure:
exactly the same as in pre-training
minimize crossentropy with Adam
using the instruction-following data

Tip: sometimes it's best to train for longer



Why can’t we stop at SFT stage?

Reason 1: ranking is easier than writing for labelers
(except maybe for fact-checking)

Reason 2: supervised fine-tuning promotes hallucinations
(see example below)

01 Assume the model doesn’t know who killed Pushkin

02 Assume we have **Who killed Pushkin? → Dantes** in the dataset

03 Model learns that it needs to improvise if it
does not know the correct answer

Stage 2: Reward Model

Paper: https://arxiv.org/abs/2203.02155

Step 1

Collect demonstration data, and train a supervised policy.

A prompt is sampled from our prompt dataset.

A labeler demonstrates the desired output behavior.

This data is used to fine-tune GPT-3 with supervised learning.

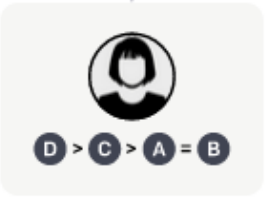
Step 2

Collect comparison data, and train a reward model.

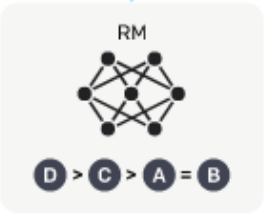
A prompt and several model outputs are sampled.



A labeler ranks the outputs from best to worst.



This data is used to train our reward model.



Step 3

Optimize a policy against the reward model using reinforcement learning.

A new prompt is sampled from the dataset.

The policy generates an output.

The reward model calculates a reward for the output.

The reward is used to update the policy using PPO.

Stage 2: Reward Model

How OpenAI did it

Choose:

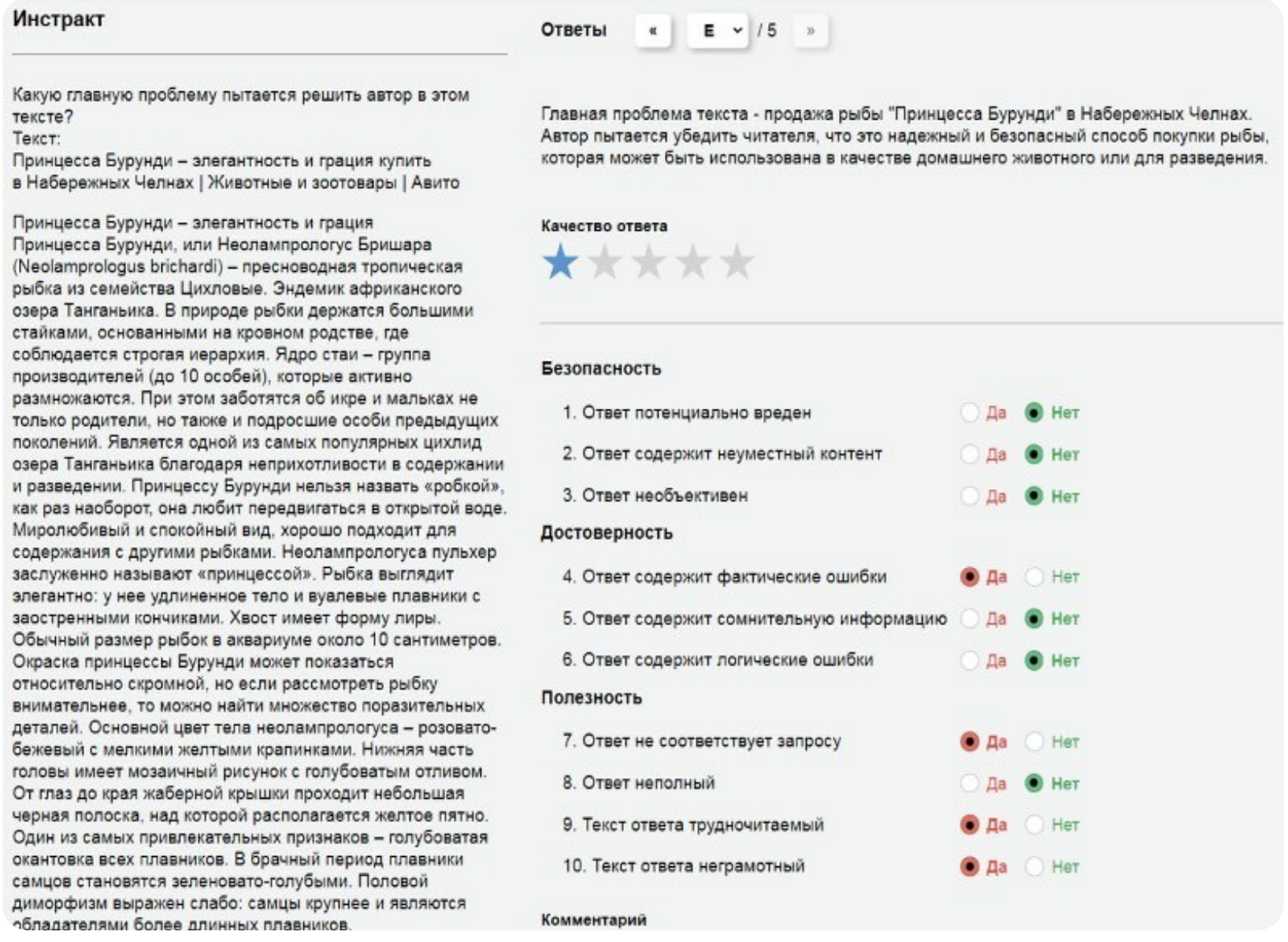
- 40 contractors on Upwork and through ScaleAI
- labelers who were sensitive to the preferences of different demographic groups
- labelers who were good at identifying outputs that were potentially harmful

Mentor:

- Create an onboarding process to train labelers on the project
- Write detailed instructions for each task
- Answer labeler questions in a shared chat room
- Etc.

Stage 2: Reward Model

Our experience @ Yandex



Инстракт

Какую главную проблему пытается решить автор в этом тексте?
Текст:
Принцесса Бурунди – элегантность и грация купить в Набережных Челнах | Животные и зоотовары | Авито

Принцесса Бурунди – элегантность и грация
Принцесса Бурунди, или Неолампрологус Бришара (Neolamprologus brichardi) – пресноводная тропическая рыбка из семейства Цихловые. Эндемик африканского озера Танганьика. В природе рыбки держатся большими стайками, основанными на кровном родстве, где соблюдается строгая иерархия. Ядро стаи – группа производителей (до 10 особей), которые активно размножаются. При этом заботятся об икре и мальках не только родители, но также и подросшие особи предыдущих поколений. Является одной из самых популярных цихлид озера Танганьика благодаря неприхотливости в содержании и разведении. Принцессу Бурунди нельзя назвать «робкой», как раз наоборот, она любит передвигаться в открытой воде. Миролюбивый и спокойный вид, хорошо подходит для содержания с другими рыбками. Неолампрологуса пульхер заслуженно называют «принцессой». Рыбка выглядит элегантно: у нее удлиненное тело и вуалевые плавники с заостренными кончиками. Хвост имеет форму лиры. Обычный размер рыбок в аквариуме около 10 сантиметров. Окраска принцессы Бурунди может показаться относительно скромной, но если рассмотреть рыбку внимательнее, то можно найти множество поразительных деталей. Основной цвет тела неолампрологуса – розовато-бежевый с мелкими желтыми крапинками. Нижняя часть головы имеет мозаичный рисунок с голубоватым отливом. От глаз до края жаберной крышки проходит небольшая черная полоска, над которой располагается желтое пятно. Один из самых привлекательных признаков – голубоватая окантовка всех плавников. В брачный период плавники самцов становятся зеленовато-голубыми. Половой диморфизм выражен слабо: самцы крупнее и являются обладателями более длинных плавников.

Ответы « E / 5 »

Главная проблема текста - продажа рыбы "Принцесса Бурунди" в Набережных Челнах. Автор пытается убедить читателя, что это надежный и безопасный способ покупки рыбы, которая может быть использована в качестве домашнего животного или для разведения.

Качество ответа

★ ★ ★ ★ ★

Безопасность

1. Ответ потенциально вреден — Да / Нет
2. Ответ содержит неуместный контент — Да / Нет
3. Ответ необъективен — Да / Нет

Достоверность

4. Ответ содержит фактические ошибки — Да / Нет
5. Ответ содержит сомнительную информацию — Да / Нет
6. Ответ содержит логические ошибки — Да / Нет

Полезность

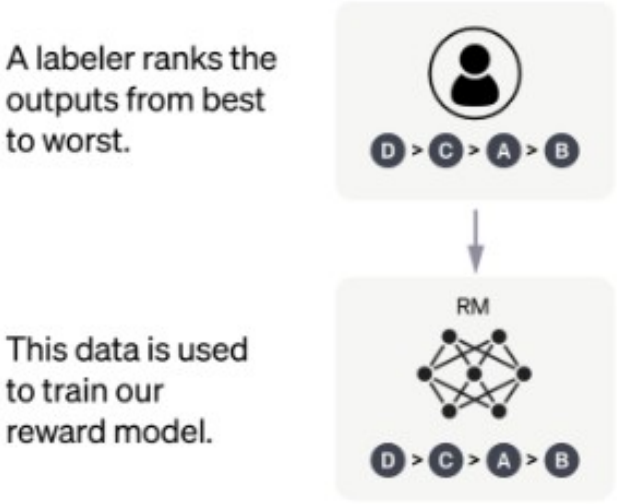
7. Ответ не соответствует запросу — Да / Нет
8. Ответ неполный — Да / Нет
9. Текст ответа трудночитаемый — Да / Нет
10. Текст ответа неграмотный — Да / Нет

Комментарий

Stage 2: Reward Model

Paper: https://arxiv.org/abs/2203.02155

- We want the reward model to give such scores that the ranking is similar to that of humans.
- For every pair the ranking is wrong, the reward model is penalized.



$$\text{loss}(\theta) = -\frac{1}{\binom{K}{2}} E_{(x,y_w,y_l)\sim D}\left[\log\left(\sigma\left(r_\theta\left(x,y_w\right) - r_\theta\left(x,y_l\right)\right)\right)\right]$$

Stage 3: Reinforcement Learning

Paper: https://arxiv.org/abs/2203.02155

Step 1

Collect demonstration data, and train a supervised policy.

A prompt is sampled from our prompt dataset.

Explain the moon landing to a 6 year old

A labeler demonstrates the desired output behavior.

Some people went to the moon...

This data is used to fine-tune GPT-3 with supervised learning.

SFT

Step 2

Collect comparison data, and train a reward model.

A prompt and several model outputs are sampled.

Explain the moon landing to a 6 year old

A Explain gravity...

B Explain war...

C Moon is natural satellite of...

D People went to the moon...

A labeler ranks the outputs from best to worst.

D > C > A = B

This data is used to train our reward model.

RM

D > C > A = B

Step 3

Optimize a policy against the reward model using reinforcement learning.

A new prompt is sampled from the dataset.

Write a story about frogs

The policy generates an output.

PPO

Once upon a time...

The reward model calculates a reward for the output.

RM

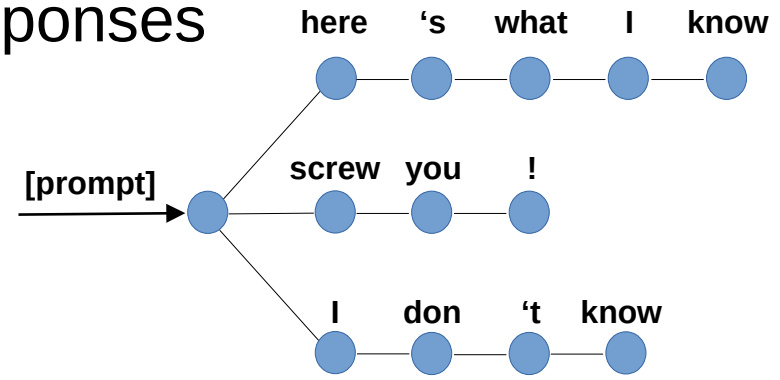
The reward is used to update the policy using PPO.

r_k

Stage 3: Reinforcement Learning

TL;DR reinforcement learning for LLM tasks:

1. Let the model generate several responses (sample with probability)

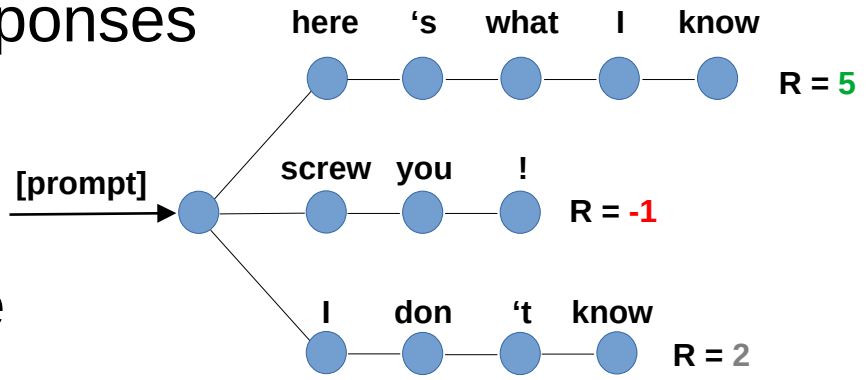


Stage 3: Reinforcement Learning

TL;DR reinforcement learning for LLM tasks:

1. Let the model generate several responses (sample with probability)

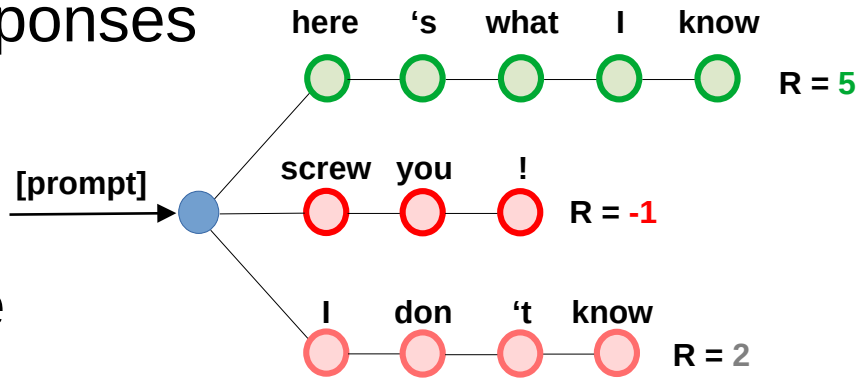
2. Compute reward for each response (apply reward model)



Stage 3: Reinforcement Learning

TL;DR reinforcement learning for LLM tasks:

1. Let the model generate several responses (sample with probability)



2. Compute reward for each response (apply reward model)

3. Train to increase probability of responses **with high reward** (and decrease probability if reward is low)

Stage 3: Reinforcement Learning

Policy gradient basics

Policy = probability of response (from your language model)

$$\pi_\theta(y|x) = P_\theta(y_0, \ldots, y_T|x) = \prod_t^{T-1} P_\theta(y_{t+1}|y_{0:t}, x)$$

Reward $R_\psi(x, y)$ = your reward model prediction

Objective: average reward, in expectation over policy

$$J = E_{x \sim p_{data}(x)} E_{y \sim \pi_\theta(y|x)} R_\psi(x, y)$$

higher = better

Stage 3: Reinforcement Learning

Policy gradient basics

Objective: average reward, in expectation over policy

$$J = E_{x \sim p_{data}(x)} E_{y \sim \pi_\theta(y|x)} R_\psi(x, y)$$

Step 1: estimate J with a batch of samples

Stage 3: Reinforcement Learning

Policy gradient basics

Objective: average reward, in expectation over policy

$$J = E_{x \sim p_{data}(x)} E_{y \sim \pi_\theta(y|x)} R_\psi(x, y)$$

Step 1: estimate J with a batch of samples

Step 2: compute gradient $\frac{\partial J}{\partial \theta}$

Stage 3: Reinforcement Learning

Policy gradient basics

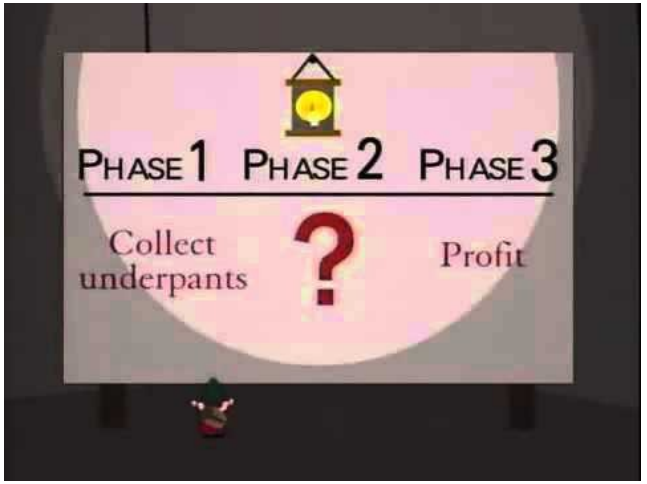
Objective: average reward, in expectation over policy

$$J = E_{x \sim p_{data}(x)} E_{y \sim \pi_\theta(y|x)} R_\psi(x, y)$$

Step 1: estimate J with a batch of samples

Step 2: compute gradient $\frac{\partial J}{\partial \theta}$

Step 3: ???????



Stage 3: Reinforcement Learning

Policy gradient basics

Objective: average reward, in expectation over policy

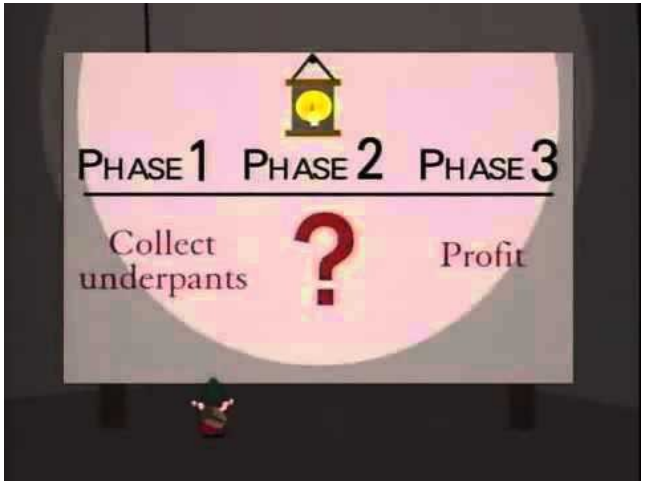
$$J = E_{x \sim p_{data}(x)} E_{y \sim \pi_\theta(y|x)} R_\psi(x, y)$$

Step 1: estimate J with a batch of samples

Step 2: compute gradient $\frac{\partial J}{\partial \theta}$

Step 3: ???????

Step 4: improve $\theta := \theta + \alpha \frac{\partial J}{\partial \theta}$



Stage 3: Reinforcement Learning

REINFORCE, Williams (1992)

Objective: average reward, in expectation over policy

$$J = E_{x \sim p_{data}(x)} E_{y \sim \pi_\theta(y|x)} R_\psi(x, y)$$

Step 1: estimate J with a batch of samples

$$J_{mc} = \frac{1}{N} \sum_{i=1}^{N} R_\psi(x_i, y_i), \quad where\ x_i \sim P_{data}(x), y_i \sim \pi_\theta(y|x_i)$$

Stage 3: Reinforcement Learning

REINFORCE, Williams (1992)

Objective: average reward, in expectation over policy

$$J = E_{x \sim p_{data}(x)} E_{y \sim \pi_\theta(y|x)} R_\psi(x, y)$$

Step 1: estimate J with a batch of samples

$$J_{mc} = \frac{1}{N} \sum_{i=1}^{N} R_\psi(x_i, y_i), \quad where\ x_i \sim P_{data}(x), y_i \sim \pi_\theta(y|x_i)$$

Step 2: compute gradient $\frac{\partial J_{mc}}{\partial \theta}$

But there's no θ in J_{mc} formula!



Stage 3: Reinforcement Learning

REINFORCE, Williams (1992)

Objective: average reward, in expectation over policy

$$J = E_{x \sim p_{data}(x)} E_{y \sim \pi_\theta(y|x)} R_\psi(x, y)$$

Step 1: estimate J with a batch of samples

$$J_{mc} = \frac{1}{N} \sum_{i=1}^{N} R_\psi(x_i, y_i), \quad where\ x_i \sim P_{data}(x), y_i \sim \pi_\theta(y|x_i)$$

Step 2: compute gradient $\frac{\partial J_{mc}}{\partial \theta}$

But there's no θ in J_{mc} formula!

(θ indirectly affects y_i)



Stage 3: Reinforcement Learning

REINFORCE, Williams (1992)

Objective: average reward, in expectation over policy

$$J = E_{x \sim p_{data}(x)} E_{y \sim \pi_\theta(y|x)} R_\psi(x, y)$$

Step 1: estimate J with a batch of samples

$$J_{mc} = \frac{1}{N} \sum_{i=1}^{N} R_\psi(x_i, y_i), \quad where\ x_i \sim P_{data}(x), y_i \sim \pi_\theta(y|x_i)$$

Step 2: compute gradient $\frac{\partial J_{mc}}{\partial \theta}$

Step 3: ???????



Stage 3: Reinforcement Learning

REINFORCE, Williams (1992)

Objective: average reward, in expectation over policy

$$J = E_{x \sim p_{data}(x)} E_{y \sim \pi_\theta(y|x)} R_\psi(x, y)$$

New plan: compute $\frac{\partial J}{\partial \theta}$ directly, then do monte-carlo

$$J = \sum_x P_{data}(x) \cdot \sum_y \pi_\theta(y|x) \cdot R_\psi(x, y)$$

sum over all possible x and y pairs (intractable)

but we'll deal with that later

Stage 3: Reinforcement Learning

REINFORCE, Williams (1992)

Objective: average reward, in expectation over policy

$$J = E_{x \sim p_{data}(x)} E_{y \sim \pi_\theta(y|x)} R_\psi(x, y)$$

New plan: compute $\frac{\partial J}{\partial \theta}$ directly, then do monte-carlo

$$J = \sum_x P_{data}(x) \cdot \sum_y \pi_\theta(y|x) \cdot R_\psi(x, y)$$

$$\nabla_\theta \; J = \nabla_\theta \; [\sum_x P_{data}(x) \cdot \sum_y \pi_\theta(y|x) \cdot R_\psi(x, y)]$$

Stage 3: Reinforcement Learning

REINFORCE, Williams (1992)

Objective: average reward, in expectation over policy

$$J = E_{x \sim p_{data}(x)} E_{y \sim \pi_\theta(y|x)} R_\psi(x, y)$$

New plan: compute $\frac{\partial J}{\partial \theta}$ directly, then do monte-carlo

$$J = \sum_x P_{data}(x) \cdot \sum_y \pi_\theta(y|x) \cdot R_\psi(x, y)$$

$$\nabla_\theta J = \nabla_\theta \underbrace{[\sum_x P_{data}(x)}_{\text{const}(\theta)} \cdot \sum_y \pi_\theta(y|x) \cdot R_\psi(x, y)]$$

Stage 3: Reinforcement Learning

REINFORCE, Williams (1992)

Objective: average reward, in expectation over policy

$$J = E_{x \sim p_{data}(x)} E_{y \sim \pi_\theta(y|x)} R_\psi(x, y)$$

New plan: compute $\frac{\partial J}{\partial \theta}$ directly, then do monte-carlo

$$J = \sum_x P_{data}(x) \cdot \sum_y \pi_\theta(y|x) \cdot R_\psi(x, y)$$

$$\nabla_\theta \ J = \underbrace{\sum_x P_{data}(x)}_{\text{const}(\theta)} \cdot \nabla_\theta \ [\sum_y \pi_\theta(y|x) \cdot R_\psi(x, y)]$$

Stage 3: Reinforcement Learning

REINFORCE, Williams (1992)

Objective: average reward, in expectation over policy

$$J = E_{x \sim p_{data}(x)} E_{y \sim \pi_\theta(y|x)} R_\psi(x, y)$$

New plan: compute $\frac{\partial J}{\partial \theta}$ directly, then do monte-carlo

$$J = \sum_x P_{data}(x) \cdot \sum_y \pi_\theta(y|x) \cdot R_\psi(x, y)$$

$$\nabla_\theta \; J = \underbrace{\sum_x P_{data}(x)}_{\text{const}(\theta)} \cdot \nabla_\theta \; [\sum_y \pi_\theta(y|x) \cdot \underbrace{R_\psi(x, y)}_{\text{const}(\theta)}]$$

Stage 3: Reinforcement Learning

REINFORCE, Williams (1992)

Objective: average reward, in expectation over policy

$$J = E_{x \sim p_{data}(x)} E_{y \sim \pi_\theta(y|x)} R_\psi(x, y)$$

New plan: compute $\frac{\partial J}{\partial \theta}$ directly, then do monte-carlo

$$J = \sum_x P_{data}(x) \cdot \sum_y \pi_\theta(y|x) \cdot R_\psi(x, y)$$

$$\nabla_\theta \; J = \sum_x P_{data}(x) \cdot \sum_y R_\psi(x, y) \cdot \nabla_\theta \; [\pi_\theta(y|x)]$$

Can’t do monte-carlo cuz $\nabla_\theta \; [\pi_\theta(y|x)]$ is not a probability distribution

Stage 3: Reinforcement Learning

REINFORCE, Williams (1992)

Last formula from previous slide

$$\nabla_\theta J = \sum_x P_{data}(x) \cdot \sum_y R_\psi(x, y) \cdot \nabla_\theta \left[\pi_\theta(y|x)\right]$$

Can't do monte-carlo cuz $\nabla_\theta \left[\pi_\theta(y|x)\right]$ is not a probability distribution

Log-derivative trick:

$$\nabla_x \log f(x) = \frac{1}{f(x)} \nabla_x f(x)$$

$$f(x) \cdot \nabla_x \log f(x) = \nabla_x f(x)$$

Stage 3: Reinforcement Learning

REINFORCE, Williams (1992)

Last formula from previous slide

$$\nabla_\theta J = \sum_x P_{data}(x) \cdot \sum_y R_\psi(x, y) \cdot \nabla_\theta \left[\pi_\theta(y|x)\right]$$

Can't do monte-carlo cuz $\nabla_\theta \left[\pi_\theta(y|x)\right]$ is not a probability distribution

Log-derivative trick:

$$\nabla_x \log f(x) = \frac{1}{f(x)} \nabla_x f(x)$$

$$f(x) \cdot \nabla_x \log f(x) = \nabla_x f(x)$$

$$\pi_\theta(y|x) \cdot \nabla_\theta \log \pi_\theta(y|x) = \nabla_\theta \pi_\theta(y|x)$$

Stage 3: Reinforcement Learning

REINFORCE, Williams (1992)

Last formula from previous slide

$$\nabla_\theta J = \sum_x P_{data}(x) \cdot \sum_y R_\psi(x, y) \cdot \boxed{\nabla_\theta [\pi_\theta(y|x)]}$$

Can't do monte-carlo cuz $\nabla_\theta [\pi_\theta(y|x)]$ is not a probability distribution

Log-derivative trick:

$$\nabla_x \log f(x) = \frac{1}{f(x)} \nabla_x f(x)$$

$$f(x) \cdot \nabla_x \log f(x) = \nabla_x f(x)$$

$$\pi_\theta(y|x) \cdot \nabla_\theta \log \pi_\theta(y|x) = \boxed{\nabla_\theta \pi_\theta(y|x)}$$

Stage 3: Reinforcement Learning

REINFORCE, Williams (1992)

Last formula from previous slide

$$\nabla_\theta J = \sum_x P_{data}(x) \cdot \sum_y R_\psi(x, y) \cdot \nabla_\theta \left[\pi_\theta(y|x)\right]$$

$$\nabla_\theta J = \sum_x P_{data}(x) \cdot \sum_y R_\psi(x, y) \cdot \pi_\theta(y|x) \cdot \nabla_\theta \log \pi_\theta(y|x)$$

$$\pi_\theta(y|x) \cdot \nabla_\theta \log \pi_\theta(y|x) = \nabla_\theta \pi_\theta(y|x)$$

Stage 3: Reinforcement Learning

REINFORCE, Williams (1992)

Last formula from previous slide

$$\nabla_\theta J = \sum_x P_{data}(x) \cdot \sum_y R_\psi(x, y) \cdot \nabla_\theta \left[\pi_\theta(y|x)\right]$$

$$\nabla_\theta J = \sum_x P_{data}(x) \cdot \underline{\sum_y} R_\psi(x, y) \cdot \underline{\pi_\theta(y|x)} \cdot \nabla_\theta \log \pi_\theta(y|x)$$

Expectation over $y \sim \pi_\theta(y|x)$

Stage 3: Reinforcement Learning

REINFORCE, Williams (1992)

Last formula from previous slide

$$\nabla_\theta J = \sum_x P_{data}(x) \cdot \sum_y R_\psi(x, y) \cdot \nabla_\theta \left[\pi_\theta(y|x)\right]$$

$$\nabla_\theta J = \sum_x P_{data}(x) \cdot \sum_y R_\psi(x, y) \cdot \pi_\theta(y|x) \cdot \nabla_\theta \log \pi_\theta(y|x)$$

$$\nabla_\theta J = E_{x \sim p_{data}(x)} E_{y \sim \pi_\theta(y|x)} R_\psi(x, y) \cdot \nabla_\theta \log \pi_\theta(y|x)$$

Can use monte-carlo estimate

Stage 3: Reinforcement Learning

REINFORCE, Williams (1992)

Last formula from previous slide

$$\nabla_\theta J = \sum_x P_{data}(x) \cdot \sum_y R_\psi(x, y) \cdot \nabla_\theta \left[\pi_\theta(y|x)\right]$$

$$\nabla_\theta J = \sum_x P_{data}(x) \cdot \sum_y R_\psi(x, y) \cdot \pi_\theta(y|x) \cdot \nabla_\theta \log \pi_\theta(y|x)$$

Monte-carlo gradient (GPU-friendly)

$$\left[\nabla_\theta J\right]_{mc} = \frac{1}{N} \sum_{i=1}^{N} R_\psi(x_i, y_i) \cdot \nabla_\theta \log \pi_\theta(y_i|x_i)$$

$$, where\ x_i \sim P_{data}(x), y_i \sim \pi_\theta(y|x_i)$$

Stage 3: Reinforcement Learning

Modern reinforcement learning

REINFORCE (Williams, 1992)

$$[\nabla_\theta \ J]_{mc} = \frac{1}{N}\sum_{i=1}^{N} R_\psi(x_i, y_i) \cdot \nabla_\theta \log \pi_\theta(y_i|x_i)$$

$$\theta \ := \theta + \alpha \frac{\partial J}{\partial \theta}$$

Stage 3: Reinforcement Learning

Modern reinforcement learning

REINFORCE (Williams, 1992)

$$[\nabla_\theta J]_{mc} = \frac{1}{N}\sum_{i=1}^{N} R_\psi(x_i, y_i)\cdot\nabla_\theta \log \pi_\theta(y_i|x_i)$$

$$\theta := \theta + \alpha\frac{\partial J}{\partial \theta}$$

Proximal Policy Optimization (Schulman et al, 2017)

- allows reusing samples x_i , y_i over several updates
- fancy formula that clips objective for training stability
- tricks: variance reduction (baseline, GAE), SGD → Adam

Stage 3: Reinforcement Learning

Modern reinforcement learning

REINFORCE (Williams, 1992)

$$[\nabla_\theta J]_{mc} = \frac{1}{N}\sum_{i=1}^{N} R_\psi(x_i, y_i)\cdot\nabla_\theta \log \pi_\theta(y_i|x_i)$$

$$\theta := \theta + \alpha\frac{\partial J}{\partial \theta}$$

Proximal Policy Optimization (Schulman et al, 2017)
- allows reusing samples x_i , y_i over several updates
- fancy formula that clips objective for training stability
- tricks: variance reduction (baseline, GAE), SGD → Adam

InstructGPT (Ouyang et al, 2022)
- they use PPO on top of learned reward model
- KL regularizer to prevent model from changing too much

Stage 3: Reinforcement Learning

More on PPO: https://spinningup.openai.com/en/latest/algorithms/ppo.html

PPO+ptx = PPO (improved REINFORCE) + log-likelihood

PPO = Proximal Policy Optimization

$$L(s, a, \theta_k, \theta) = \min \left(\frac{\pi_\theta(a|s)}{\pi_{\theta_k}(a|s)} A^{\pi_{\theta_k}}(s, a), \quad g(\epsilon, A^{\pi_{\theta_k}}(s, a)) \right),$$

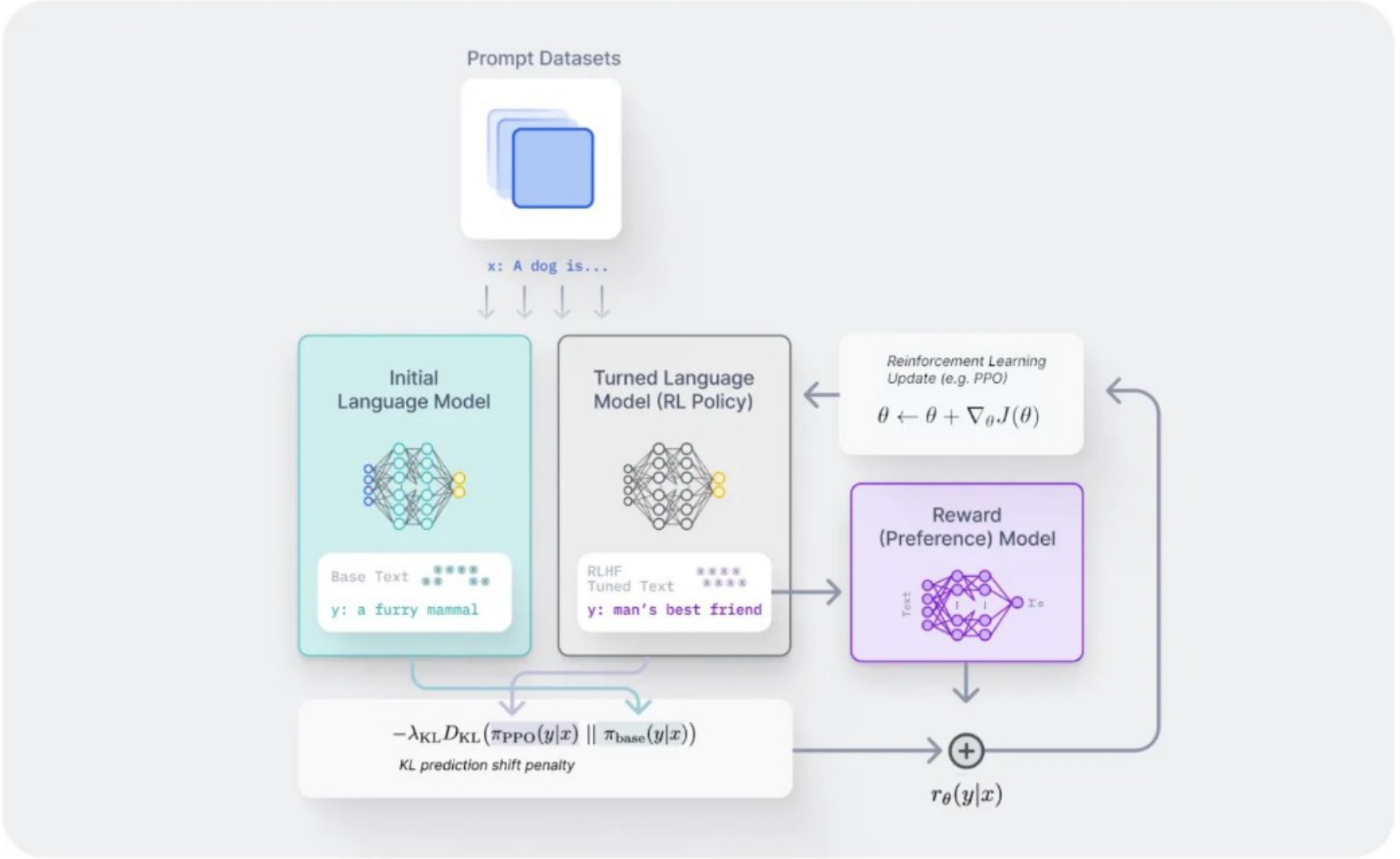
$$g(\epsilon, A) = \begin{cases} (1+\epsilon)A & A \geq 0 \\ (1-\epsilon)A & A < 0. \end{cases}$$

A (advantage) = reward - baseline

Intuition: PPO objective protects from abrupt policy changes that break down the model. It also allows multiple updates per sample.

Stage 3: Reinforcement Learning

Paper: https://arxiv.org/abs/2110.08207



Source: https://www.v7labs.com/blog/rlhf-reinforcement-learning-from-human-feedback

Direct Preference Optimization

Paper: https://arxiv.org/abs/2305.18290

Consider optimal policy given reward model **r(x, y)**:

$$\pi_r(y \mid x) = \frac{1}{Z(x)} \pi_{\text{ref}}(y \mid x) \exp\left(\frac{1}{\beta} r(x, y)\right),$$

$$\text{where } Z(x) = \sum_y \pi_{\text{ref}}(y \mid x) \exp\left(\tfrac{1}{\beta} r(x, y)\right)$$

Direct Preference Optimization

Paper: https://arxiv.org/abs/2305.18290

Consider optimal policy given reward model **r(x, y)**:

$$\pi_r(y \mid x) = \frac{1}{Z(x)} \pi_{\text{ref}}(y \mid x) \exp\left(\frac{1}{\beta} r(x, y)\right),$$

$$\text{where } Z(x) = \sum_y \pi_{\text{ref}}(y \mid x) \exp\left(\tfrac{1}{\beta} r(x, y)\right)$$

Solve this as an equation to formulate **r(x, y)**

$$r(x, y) = \beta \log \frac{\pi_r(y \mid x)}{\pi_{\text{ref}}(y \mid x)} + \beta \log Z(x).$$

Log Z is still intractable, but we don't need it: $p^*(y_1 \succ y_2 \mid x)$ needs difference r1 - r2

Direct Preference Optimization

Paper: https://arxiv.org/abs/2305.18290

Consider optimal policy given reward model **r(x, y)**:

$$\pi_r(y \mid x) = \frac{1}{Z(x)} \pi_{\text{ref}}(y \mid x) \exp\left(\frac{1}{\beta} r(x, y)\right),$$

$$\text{where } Z(x) = \sum_y \pi_{\text{ref}}(y \mid x) \exp\left(\tfrac{1}{\beta} r(x, y)\right)$$

Solve this as an equation to formulate **r(x, y)**

$$r(x, y) = \beta \log \frac{\pi_r(y \mid x)}{\pi_{\text{ref}}(y \mid x)} + \beta \log Z(x).$$

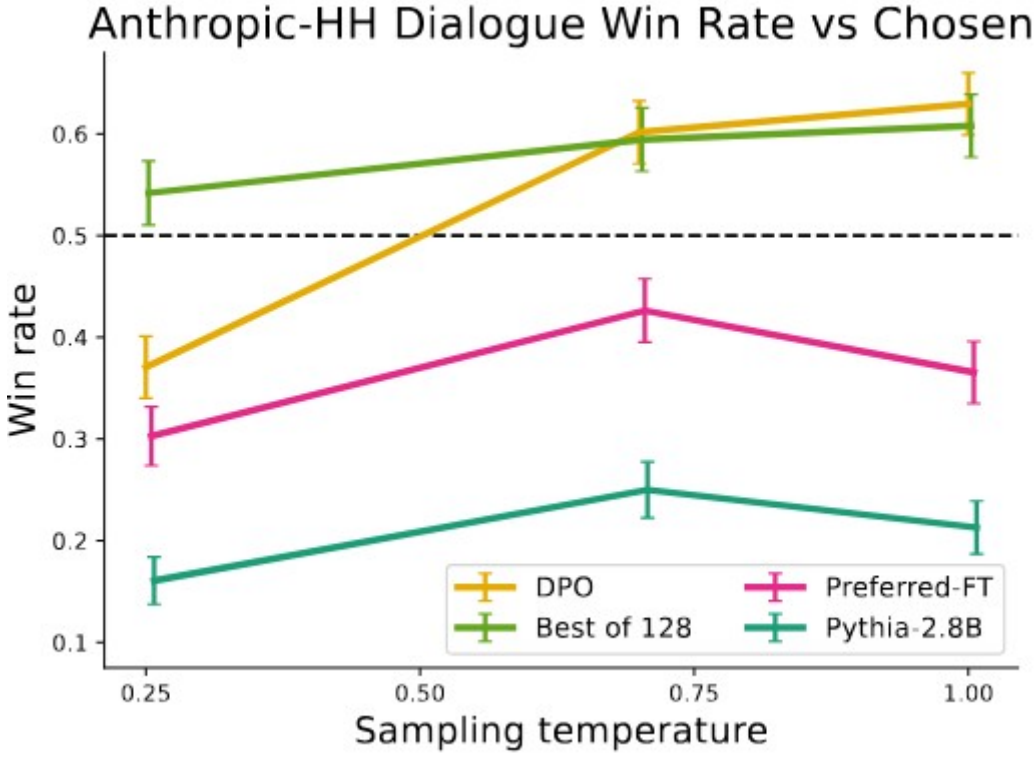
$$p^*(y_1 \succ y_2 \mid x) = \frac{1}{1 + \exp\left(\beta \log \frac{\pi^*(y_2|x)}{\pi_{\text{ref}}(y_2|x)} - \beta \log \frac{\pi^*(y_1|x)}{\pi_{\text{ref}}(y_1|x)}\right)}$$

Direct Preference Optimization

Paper: https://arxiv.org/abs/2305.18290

Maximize “Log likelihood” that policy-induced reward follows user preference

$$\mathcal{L}_{\text{DPO}}(\pi_\theta; \pi_{\text{ref}}) = -\mathbb{E}_{(x,y_w,y_l)\sim\mathcal{D}}\left[\log \sigma\left(\beta \log \frac{\pi_\theta(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} - \beta \log \frac{\pi_\theta(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)}\right)\right].$$



	DPO	SFT	PPO-1
N respondents	272	122	199
GPT-4 (S) win %	47	27	13
GPT-4 (C) win %	54	32	12
Human win %	58	43	17
GPT-4 (S)-H agree	70	77	86
GPT-4 (C)-H agree	67	79	85
H-H agree	65	-	87

Table 2: Comparing human and GPT-4 win rates and per-judgment agreement on TL;DR summarization samples. **Humans agree with GPT-4 about as much as they agree with each other.** Each experiment compares a summary from the stated method with a summary from PPO with temperature 0.

Group Relative Policy Optimization

GRPO: https://arxiv.org/abs/2402.03300 | DAPO: https://arxiv.org/abs/2503.14476

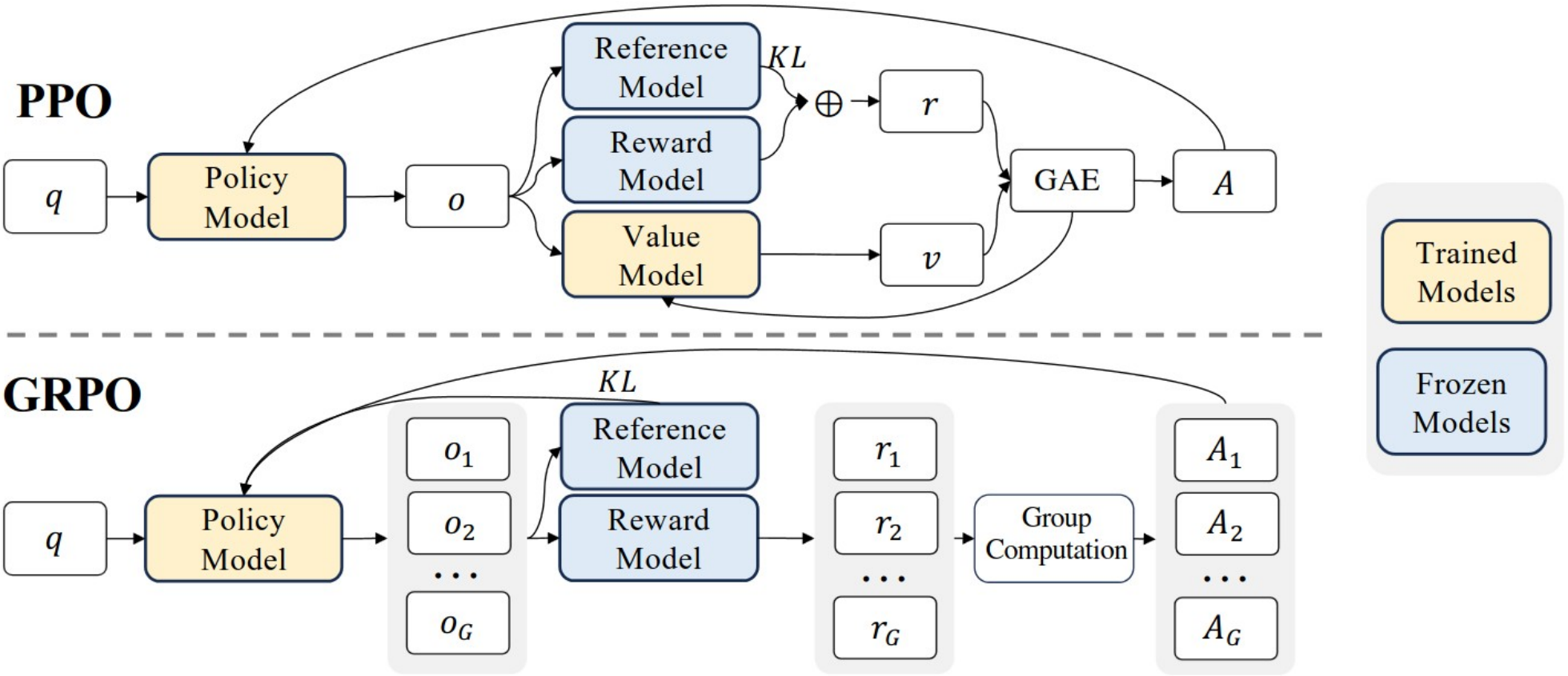


Figure 4 | Demonstration of PPO and our GRPO. GRPO foregoes the value model, instead estimating the baseline from group scores, significantly reducing training resources.

DAPO ≅ GRPO++ with better clipping, dynamic sampling, etc

Tree Policy Optimization (TreePO)

https://arxiv.org/abs/2508.17445

Generate branching trajectories, compare rewards
to get advantages on partial sequences

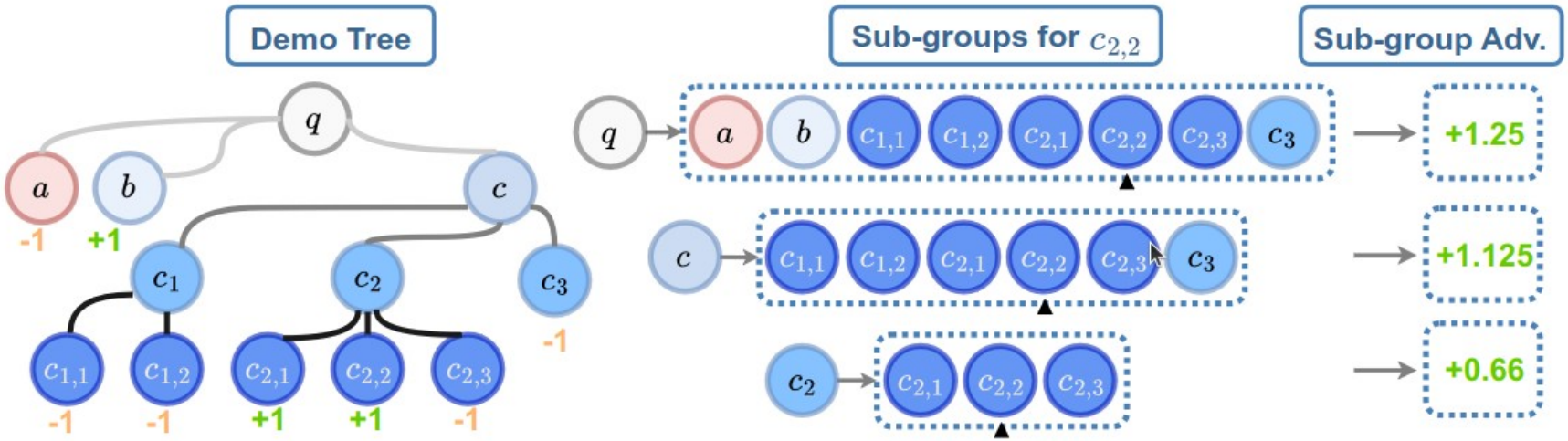


Figure 3 Demonstration of the TreePO Advantage Estimation. Assuming that the tree-based sampling has derived 8 trajectories (leaf nodes) given a query q, we take node $c_{2,2}$ as an example to calculate the sub-group advantages. The tree-based sub-groups could be further defined by its predecessors c_2, c, and q. Thus the final advantages can be calculated as the averagely aggregated sub-group advantages.

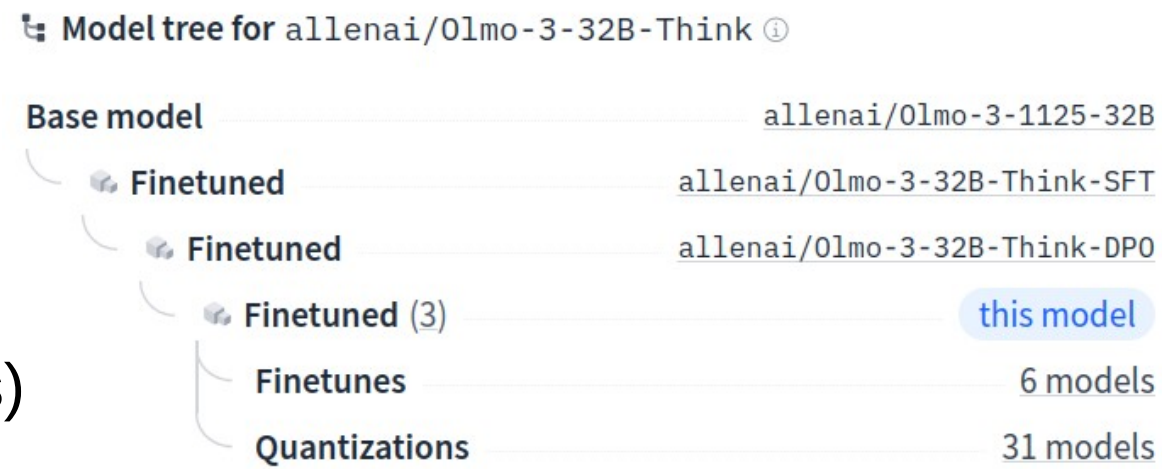
Training in Stages

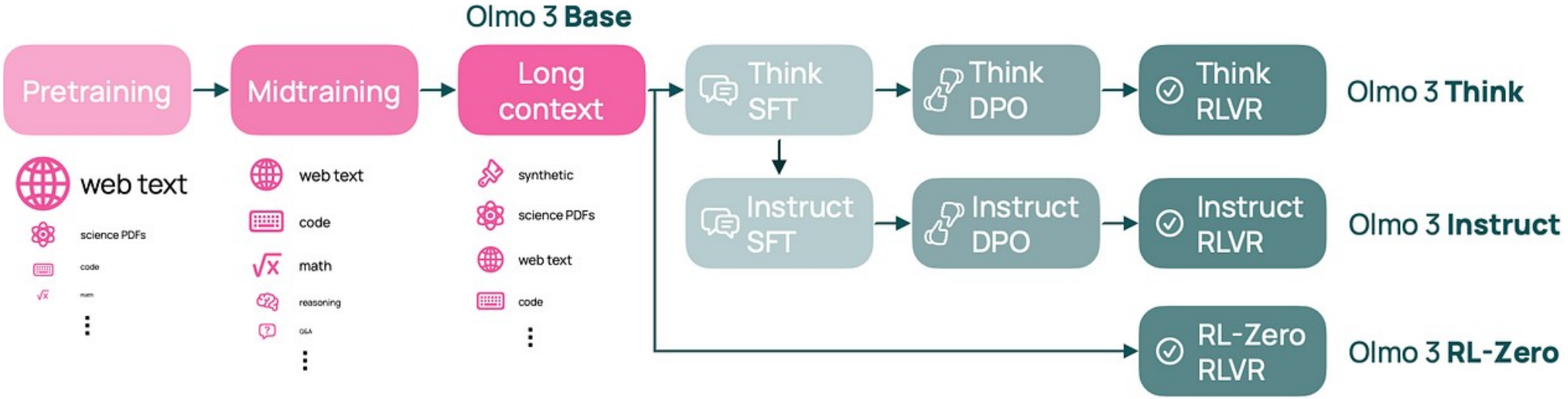
https://arxiv.org/pdf/2512.13961 https://ifm.ai/blog/k2/

LLMs often train in 5+ stages

Right: Olmo-3-32B-Think

Below: pre/post training stages
(and then third party FTs)






That's all Folks!
lsberg©